Java Backend Developer Interview Guide

Spring Core, Spring Boot, REST APIs, JDBC, Hibernate & JPA — 12-Day Study Notes

Prepared for Rachit

August 2026

Contents

How to Use This Guide **5**

Table of Contents **6**

DAY 1 — Spring Framework Core **7**
1. Course Introduction . . . 7
2. Tight Coupling and Loose Coupling [KEY] . . . 7
3. Dependency Injection [KEY] . . . 8
4. Web Framework . . . 10
5. Spring Framework . . . 10
6. Spring Core Concepts [KEY] . . . 11
7. Creating a Spring Project . . . 12
8. Build Tools . . . 12
9. Spring Bean [KEY] . . . 13
10. IoC and DI — Tying It Together . . . 14
Day 1 — Common Mistakes . . . 14
Day 1 — Interview Questions . . . 14
Day 1 — Quick Revision . . . 14

DAY 2 — Annotations & Layered Architecture **16**
1. Problems with Manual Configuration . . . 16
2. @Component [KEY] . . . 16
3. @ComponentScan . . . 16
4. @Configuration . . . 16
5. @Bean . . . 17
6. @Autowired [KEY] . . . 17
7. Stereotype Annotations [KEY] . . . 18
8. Multiple Implementations / Ambiguity [KEY] . . . 18
9. Bean Lifecycle [KEY] . . . 19
10. Layered Architecture [KEY] . . . 20
Day 2 — Common Mistakes . . . 21
Day 2 — Interview Questions . . . 21
Day 2 — Quick Revision . . . 22

DAY 3 — Spring Boot Introduction **23**
1. What is Spring Boot? [KEY] . . . 23
2. Creating a Spring Boot Project . . . 23

3. Spring Boot Starters [KEY] . . . 24
4. application.properties . . . 25
5. CommandLineRunner . . . 25
6. ApplicationRunner . . . 25
Day 3 — Common Mistakes . . . 26
Day 3 — Interview Questions (20) . . . 26
Day 3 — Quick Revision . . . 27

DAY 4 — First REST API **28**
1. REST API — Basics . . . 28
2. HTTP Methods [KEY] . . . 28
3. HTTP Status Codes [KEY] . . . 28
4. First REST API — Example . . . 29
5. Returning JSON . . . 29
6. @RequestMapping and HTTP-Specific Annotations [KEY] . . . 30
7. Internal Request Flow [KEY] (very common interview topic) . . . 30
Day 4 — Common Mistakes . . . 31
Day 4 — Interview Questions . . . 31
Day 4 — Quick Revision . . . 31

DAY 5 — POST, PUT, DELETE **32**
POST — Creating Resources . . . 32
PUT — Updating Resources . . . 32
DELETE — Deleting Resources . . . 33
ResponseEntity [KEY] . . . 33
Complete CRUD REST API (in-memory) . . . 33
Day 5 — Common Mistakes . . . 34
Day 5 — Interview Questions . . . 34
Day 5 — Quick Revision . . . 35

DAY 6 — Path Variables & Query Parameters **36**
@PathVariable [KEY] . . . 36
@RequestParam [KEY] . . . 36
@RequestHeader . . . 37
Combining Parameters — Full Example . . . 37
Day 6 — Common Mistakes . . . 37
Day 6 — Interview Questions . . . 38
Day 6 — Quick Revision . . . 38

DAY 7 — Exception Handling & Logging **39**
1. Why Exception Handling? . . . 39
2. Service Layer Delegation . . . 39
3. @ExceptionHandler . . . 39
4. @RestControllerAdvice [KEY] . . . 40
5. Custom Exceptions . . . 40
6. Standard Error Response . . . 41
7. Logging [KEY] . . . 41
Day 7 — Common Mistakes . . . 42
Day 7 — Interview Questions . . . 42
Day 7 — Quick Revision . . . 43

DAY 8 — JDBC Fundamentals **44**
1. Why JDBC? . . . 44
2. JDBC Architecture . . . 44
3. Important JDBC Classes/Interfaces [KEY] . . . 44

4. Database Connection — Complete Example . . . 44
5. try-with-resources [KEY] . . . 45
6. CRUD Using JDBC . . . 45
Day 8 — Common Mistakes . . . 46
Day 8 — Interview Questions . . . 46
Day 8 — Quick Revision . . . 47

DAY 9 — PreparedStatements & Transactions **48**
PreparedStatement [KEY] . . . 48
Transactions [KEY] . . . 48
commit(), rollback(), setAutoCommit(false) . . . 49
Day 9 — Common Mistakes . . . 50
Day 9 — Interview Questions . . . 50
Day 9 — Quick Revision . . . 51

DAY 10 — Hibernate & ORM **52**
ORM (Object-Relational Mapping) [KEY] . . . 52
Hibernate . . . 52
JPA vs Hibernate [KEY] (Very important interview topic) . . . 53
JDBC vs Hibernate [KEY] . . . 53
Hibernate CRUD . . . 54
Day 10 — Common Mistakes . . . 55
Day 10 — Interview Questions . . . 55
Day 10 — Quick Revision . . . 56

DAY 11 — Spring Boot + JPA **57**
The Layering of Persistence Technologies [KEY] . . . 57
EntityManager [KEY] . . . 57
Spring Data JPA [KEY] . . . 58
Complete Spring Boot JPA CRUD . . . 58
Day 11 — Common Mistakes . . . 61
Day 11 — Interview Questions . . . 61
Day 11 — Quick Revision . . . 61

DAY 12 — JPA Relationships **62**
Why Relationships? . . . 62
@OneToOne [KEY] . . . 62
@OneToMany / @ManyToOne [KEY] (always used as a pair) . . . 63
@ManyToMany [KEY] . . . 63
Cascade [KEY] . . . 64
Day 12 — Common Mistakes . . . 65
Day 12 — Interview Questions . . . 65
Day 12 — Quick Revision . . . 65

Master Comparison Tables [KEY] **66**

Complete Project — Employee Management REST API **71**
Architecture . . . 71
Project Structure . . . 71
application.properties . . . 71
Entities . . . 72
DTOs . . . 72
Repository . . . 73
Custom Exception . . . 73
Service . . . 73

Controller . . . 74
Global Exception Handler . . . 75
Main Class . . . 76
API Summary . . . 76

100 Spring Boot Interview Questions **77**
Beginner (30) . . . 77
Intermediate (40) . . . 80
Advanced (20) . . . 83
Scenario-Based (10) . . . 85

Rapid Revision Cheat Sheet **87**
Spring Core . . . 87
Spring Boot . . . 87
REST . . . 87
Exception Handling & Logging . . . 87
JDBC . . . 87
Hibernate/JPA . . . 88

Final Mock Interview — 50 Questions, 10 Rounds **89**
Round 1 — Basic Java/Spring . . . 89
Round 2 — Spring Core . . . 89
Round 3 — Spring Boot . . . 90
Round 4 — REST API . . . 90
Round 5 — Exception Handling . . . 90
Round 6 — JDBC . . . 91
Round 7 — Transactions . . . 91
Round 8 — Hibernate/JPA . . . 91
Round 9 — Relationships . . . 92
Round 10 — Real-World Scenarios . . . 92

How to Use This Guide

This guide takes you from basic Java knowledge to interview-confident on Spring, Spring Boot, REST APIs, JDBC, Hibernate, and JPA. Each topic follows the same pattern: **definition → why it's needed → how it works internally → example → code → real-world use → common mistakes → interview questions → quick revision.**

[KEY] marks topics that come up in almost every Java backend interview — pay extra attention to these.

Table of Contents

- Day 1 — Spring Framework Core
- Day 2 — Annotations & Layered Architecture
- Day 3 — Spring Boot Introduction
- Day 4 — First REST API
- Day 5 — POST, PUT, DELETE
- Day 6 — Path Variables & Query Parameters
- Day 7 — Exception Handling & Logging
- Day 8 — JDBC Fundamentals
- Day 9 — PreparedStatement & Transactions
- Day 10 — Hibernate & ORM
- Day 11 — Spring Boot + JPA
- Day 12 — JPA Relationships
- Master Comparison Tables (30)
- Complete Project — Employee Management REST API
- 100 Interview Questions
- Rapid Revision Cheat Sheet
- Final Mock Interview (50 Questions)

DAY 1 — Spring Framework Core

1. Course Introduction

Before Spring existed, Java developers built enterprise applications using **EJB (Enterprise Java Beans)**, which was powerful but extremely heavy — lots of boilerplate, XML configuration, and slow development. Spring was created (Rod Johnson, 2003) as a **lightweight alternative** that made Java enterprise development simpler by:

- Managing object creation and wiring for you (IoC/DI)
- Removing boilerplate through abstraction (JDBC templates, transaction management)
- Being modular — use only what you need (Core, MVC, Security, Data, etc.)

Spring became popular because it let developers focus on **business logic** instead of plumbing code (object creation, resource management, configuration).

---

2. Tight Coupling and Loose Coupling [KEY]

What is coupling?

Coupling is the degree to which one class depends on another. High dependency = tight coupling. Low dependency (depending on abstractions, not concrete classes) = loose coupling.

Tight coupling — the problem

```
class EmailService {
    void sendEmail() {
        System.out.println("Email sent");
    }
}

class NotificationService {
    private EmailService emailService = new EmailService(); // tightly coupled

    void notifyUser() {
        emailService.sendEmail();
    }
}
```

Problems with this: - `NotificationService` is permanently bound to `EmailService`. If tomorrow you need SMS notifications, you must edit `NotificationService`'s code. - Hard to unit test — you cannot substitute a mock `EmailService` easily. - Violates the **Open/Closed Principle** (code should be open for extension, closed for modification).

Loose coupling — the fix (interface-based design)

```
interface MessageService {
    void sendMessage();
}

class EmailService implements MessageService {
    public void sendMessage() {
```

```
        System.out.println("Email sent");
    }
}

class SmsService implements MessageService {
    public void sendMessage() {
        System.out.println("SMS sent");
    }
}

class NotificationService {
    private MessageService messageService;

    NotificationService(MessageService messageService) { // depends on abstraction
        this.messageService = messageService;
    }

    void notifyUser() {
        messageService.sendMessage();
    }
}
```

Why this is better: - `NotificationService` depends on the `MessageService` **interface**, not a specific implementation. - You can pass in `EmailService`, `SmsService`, or a mock — without touching `NotificationService`. - This is the foundation Spring's Dependency Injection is built on.

Real-world analogy: A wall socket (interface) accepts any plug (implementation) that follows the standard shape — you don't rewire your house for every new appliance.

---

3. Dependency Injection [KEY]

What is a dependency?

If class A uses class B, then B is a **dependency** of A.

What is Dependency Injection (DI)?

Instead of a class **creating** its own dependencies (`new EmailService()`), an external source (the Spring container) **creates and supplies (injects)** them.

Why DI is required

- Removes tight coupling
- Makes classes easily testable (inject mocks)
- Centralizes object creation and configuration
- Supports the **Dependency Inversion Principle** (depend on abstractions)

Types of Injection

1. Constructor Injection (recommended [KEY])

```
@Component
class NotificationService {
    private final MessageService messageService;

    @Autowired
    NotificationService(MessageService messageService) {
        this.messageService = messageService;
    }
}
```

2. Setter Injection

```
@Component
class NotificationService {
    private MessageService messageService;

    @Autowired
    public void setMessageService(MessageService messageService) {
        this.messageService = messageService;
    }
}
```

3. Field Injection (not recommended)

```
@Component
class NotificationService {
    @Autowired
    private MessageService messageService;
}
```

Which is recommended, and why? [KEY]

Constructor injection is preferred because: - Dependencies can be made `final` (immutable, thread-safe) - Object is created in a fully-initialized state — no `NullPointerException` risk - Makes mandatory dependencies explicit (constructor won't compile without them) - Easy to unit test without a Spring container (just call `new NotificationService(mockService)`) - Avoids circular dependency issues silently — Spring fails fast at startup instead of at runtime

Field injection is discouraged because it hides dependencies, requires reflection (slower, less safe), and makes classes hard to test outside Spring.

Dependency Injection vs Dependency Lookup

Aspect	Dependency Injection	Dependency Lookup
Who initiates	Container pushes dependency into the object	Object actively asks/searches for the dependency (e.g., JNDI lookup, `context.getBean()`)
Coupling	Loosely coupled to container	Coupled to lookup mechanism
Modern usage	Standard in Spring	Rarely used directly in modern Spring apps

4. Web Framework

What is a framework?

A framework is reusable, pre-written code that provides structure and common functionality, so you build applications by filling in specific logic rather than writing everything from scratch.

Framework vs Library [KEY]

Aspect	Library	Framework
Control flow	Your code calls the library	Framework calls your code ("Inversion of Control")
Flexibility	You decide when/how to use it	Framework dictates the structure
Example	A JSON parsing utility	Spring, Django, Angular

Analogy: A library is a toolbox you pick tools from. A framework is a factory assembly line you plug your work into.

What is a web framework?

A framework specialized for building web applications — handling HTTP requests/responses, routing, sessions, and often database access (e.g., Spring MVC, Django, Express).

Why frameworks are needed

- Avoid reinventing common infrastructure (routing, security, validation)
- Enforce consistent architecture across a team
- Faster development, fewer bugs, easier maintenance

---

5. Spring Framework

What is Spring?

Spring is a comprehensive framework for building Java applications, built around **Inversion of Control (IoC)** and **Dependency Injection (DI)** as its core, with many modules for different concerns (web, data access, security).

Problems Spring solves

- Tight coupling between classes
- Manual, repetitive object creation and wiring
- Boilerplate for JDBC, transactions, and exception handling
- Difficulty testing due to hard dependencies

Spring Modules / Ecosystem

```
                    Spring Framework
                           |
```

```
   ------------------------------------------------
   |            |            |            |            |
Spring Core  Spring MVC  Spring JDBC  Spring ORM   Spring AOP
   |
   ----------------------------
   |                          |
Spring Data                Spring Security
```

- **Spring Core** — IoC container, Beans, DI (foundation of everything else)
- **Spring MVC** — Web layer: `DispatcherServlet`, controllers, REST support
- **Spring JDBC** — Simplifies raw JDBC (JdbcTemplate removes boilerplate)
- **Spring ORM** — Integration with Hibernate/JPA
- **Spring Data** — Repository abstraction (Spring Data JPA, MongoDB, etc.)
- **Spring Security** — Authentication and authorization
- **Spring AOP** — Cross-cutting concerns (logging, transactions) via proxies

---

6. Spring Core Concepts [KEY]

- **IoC (Inversion of Control):** A design principle where control of object creation and lifecycle is handed from your code to a container/framework.
- **DI (Dependency Injection):** The mechanism through which IoC is implemented in Spring — the container injects dependencies into objects.
- **Spring Container:** The engine that creates, configures, and manages the lifecycle of beans. Comes in two flavors: `BeanFactory` and `ApplicationContext`.
- **Bean:** Any object that is created, configured, and managed by the Spring container.
- **ApplicationContext:** The advanced, commonly-used Spring container — adds event handling, internationalization, AOP integration, and eager bean loading on top of `BeanFactory`.
- **BeanFactory:** The basic, lightweight container — lazy-loads beans, minimal features. Rarely used directly today.
- **Configuration:** Metadata (annotations, Java config classes, or XML) that tells Spring what beans to create and how to wire them.
- **Component Scanning:** The process where Spring scans specified packages for classes annotated with stereotypes (`@Component`, `@Service`, etc.) and registers them as beans automatically.

The relationship

```
Spring Application
       ↓
Spring Container (ApplicationContext)
       ↓
Reads Configuration (annotations / Java config)
       ↓
Creates Beans
       ↓
Resolves & Injects Dependencies
       ↓
Application runs using fully-wired objects
```

---

7. Creating a Spring Project

A typical Spring project (Maven-based) looks like:

```
my-app/
├── pom.xml                    ← dependencies & build config
├── src/main/java/              ← application source code
│   └── com/example/app/
│       ├── AppConfig.java      ← configuration class
│       └── MainApp.java        ← entry point
└── src/main/resources/         ← properties, XML configs
```

- **Main class:** contains `public static void main(String[] args)`, which bootstraps the `ApplicationContext`.
- **Configuration:** either a `@Configuration` Java class or (in legacy projects) XML.

---

8. Build Tools

Maven

What is Maven? A build automation and dependency management tool for Java projects, driven by an XML file called `pom.xml` (Project Object Model).

Why Maven? - Automatically downloads and manages library JARs (and their versions) from repositories - Standardizes project structure across teams - Defines a repeatable build lifecycle

pom.xml basics:

```xml
<dependencies>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>6.1.0</version>
    </dependency>
</dependencies>
```

Maven lifecycle & common commands:

Command	Purpose
`mvn clean`	Deletes the `target/` build output directory
`mvn compile`	Compiles source code
`mvn test`	Runs unit tests
`mvn package`	Packages code into a JAR/WAR
`mvn install`	Installs the package into the local `.m2` repository so other local projects can use it

Gradle

What is Gradle? A newer build tool using Groovy/Kotlin scripts (`build.gradle`) instead of XML — faster builds (incremental compilation, caching) and more flexible scripting.

Maven vs Gradle

Aspect	Maven	Gradle
Config format	XML (`pom.xml`)	Groovy/Kotlin DSL (`build.gradle`)
Build speed	Slower (no build cache by default)	Faster (incremental builds, caching)
Flexibility	Convention-heavy, less flexible	Highly customizable/scriptable
Learning curve	Easier for beginners	Steeper, but more powerful
Used by	Traditional enterprise Java, most Spring tutorials	Android, many modern Java/Kotlin projects

---

9. Spring Bean [KEY]

What is a Bean?

A **Bean** is simply a Java object that is instantiated, assembled, and managed by the Spring IoC container — as opposed to an object you create yourself with `new`.

How a Bean is created

1. Spring reads configuration (annotations or Java config)
2. It identifies classes to manage (via component scanning or explicit `@Bean` methods)
3. It creates instances of these classes, resolves their dependencies, and injects them
4. It stores the fully-configured object in the container, ready to be used

Bean Lifecycle (brief preview — full detail on Day 2)

```
Definition → Instantiation → Dependency Injection → Initialization → Ready → Destruction
```

Bean Scopes

Scope	Meaning
singleton (default)	One shared instance per Spring container
prototype	A new instance is created every time the bean is requested
request	One instance per HTTP request (web apps only)
session	One instance per HTTP session (web apps only)

```java
@Component
@Scope("prototype")
public class ReportGenerator { }
```

Interview trap [KEY]: Singleton in Spring is *not* the same as the classic Gang-of-Four Singleton pattern (one instance per JVM). Spring's singleton means one instance **per container (ApplicationContext)** — a JVM can host multiple containers, each with its own "singleton" bean.

---

10. IoC and DI — Tying It Together

```
WITHOUT SPRING:
Developer → creates object (new) → manually wires dependencies → manages lifecycle

WITH SPRING:
Developer → declares what is needed (annotations/config)
Spring Container → creates the object
Spring Container → resolves and injects dependencies
Spring Container → manages the full lifecycle
```

Problem Spring solves: Without a framework, as an application grows, manually wiring dozens/hundreds of interdependent objects becomes unmanageable and error-prone. Spring's IoC container takes over this responsibility completely, letting you focus purely on business logic.

Day 1 — Common Mistakes

- Creating objects manually (`new`) inside `@Component` classes instead of injecting dependencies
- Using field injection everywhere out of habit
- Confusing a POJO with a "Bean" — a POJO only becomes a Bean once Spring manages it
- Assuming singleton scope means thread-safety automatically (it doesn't — shared mutable state in a singleton bean is a common bug source)

Day 1 — Interview Questions

Basic 1. **What is Spring Framework?** — A lightweight Java framework built around IoC/DI that simplifies enterprise application development. 2. **What is IoC?** — A principle where object creation/wiring control is transferred from application code to a container. 3. **What is a Bean?** — An object managed by the Spring IoC container.

Intermediate 4. **Why is constructor injection preferred over field injection?** — Immutability, testability, fail-fast on missing dependencies, no reflection needed. 5. **BeanFactory vs ApplicationContext — when would you use BeanFactory?** — Almost never in modern apps; ApplicationContext is used because it eager-loads beans and provides more enterprise features.

Scenario-based 6. **You have two classes, A and B, and B changes frequently. How do you design A so it isn't affected by B's changes?** — Make A depend on an interface, not B directly (loose coupling / DI).

Tricky 7. **Is every Java object in a Spring app a Bean?** — No. Only objects registered with and managed by the Spring container are Beans; objects you create with `new` yourself are plain POJOs, even inside a Spring app.

Day 1 — Quick Revision

- Coupling: tight = depends on concrete class; loose = depends on interface
- DI types: constructor (preferred), setter, field (avoid)

- Framework calls your code (IoC); library is called by your code
- Spring modules: Core, MVC, JDBC, ORM, Data, Security, AOP
- Bean = object managed by Spring container
- Default bean scope = singleton (per container, not per JVM)
- Maven = build tool + dependency manager, driven by `pom.xml`
- IoC container creates, wires, and manages the full lifecycle of beans

DAY 2 — Annotations & Layered Architecture

1. Problems with Manual Configuration

Before annotations, Spring beans were defined entirely in XML:

```
<bean id="paymentService" class="com.example.PaymentService"/>
```

For a large application with hundreds of classes, this meant hundreds of verbose XML entries — hard to read, easy to break, and disconnected from the actual code (renaming a class meant remembering to update XML too). Annotations solve this by letting configuration live **next to the code it describes**.

---

2. @Component [KEY]

Purpose: Marks a class as a Spring-managed bean — a generic stereotype for "let Spring manage this object."

How Spring detects it: During startup, Spring's component scanner walks through the configured base package(s), inspects every class, and registers any class annotated with `@Component` (or a meta-annotation built on it) as a bean definition.

```
@Component
public class PaymentService {
}
```

Bean creation: Once detected, Spring creates an instance of `PaymentService` (using its constructor, resolving any dependencies), and stores it in the container under a default name (`paymentService` — class name with the first letter lowercased).

---

3. @ComponentScan

Purpose: Tells Spring which package(s) to scan for `@Component`-annotated classes.

```
@Configuration
@ComponentScan(basePackages = "com.example.app")
public class AppConfig { }
```

- **Default behavior (Spring Boot):** `@SpringBootApplication` automatically scans the package of the main class and all its sub-packages — this is why Spring Boot projects conventionally put the main class at the root package.
- **Custom scanning:** You can override this with `basePackages` if beans live outside the default package tree.

---

4. @Configuration

Marks a class as a source of bean definitions — the modern, Java-based replacement for XML configuration files.

```
@Configuration
public class AppConfig {
```

```
    // @Bean methods go here
}
```

Spring treats `@Configuration` classes specially: it proxies them (via CGLIB) so that calling one @Bean method from another still returns the *same* singleton instance rather than creating a new object.

---

5. @Bean

Purpose: Manually declares a bean inside a `@Configuration` class — used when you don't own the class's source code (e.g., a third-party library) so you can't add `@Component` to it.

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

@Bean vs @Component

Aspect	@Component	@Bean
Applied to	Class	Method (inside a `@Configuration` class)
Use case	Your own classes	Third-party / external classes you can't annotate
Detection	Component scanning	Explicit method call
Control over instantiation	Low (Spring calls the constructor)	High (you write the object-creation logic)

---

6. @Autowired [KEY]

Tells Spring to automatically resolve and inject a matching bean.

```
@Component
class NotificationService {
    private final MessageService messageService;

    @Autowired // optional on a single constructor since Spring 4.3
    NotificationService(MessageService messageService) {
        this.messageService = messageService;
    }
}
```

- **Automatic resolution:** Spring looks for a bean of the required type in the container; if exactly one match exists, it injects it.
- Works with constructor, setter, or field injection (constructor preferred — see Day 1).

7. Stereotype Annotations [KEY]

Annotation	Layer	Purpose
`@Component`	Generic	Base stereotype — any Spring-managed class
`@Service`	Service	Marks business-logic classes (semantic clarity)
`@Repository`	Data access (DAO)	Marks data-access classes; **also** enables automatic translation of database exceptions into Spring's unified `DataAccessException` hierarchy
`@Controller`	Web	Marks a class that handles web requests and returns view names
`@RestController`	Web (REST)	`@Controller` + `@ResponseBody` combined — returns data (JSON) directly instead of a view name

Are `@Service`/`@Repository` technically different from `@Component`? Structurally, `@Service` and `@Controller` are just `@Component` under the hood (meta-annotated with `@Component`) — functionally identical for bean registration. `@Repository` is the one exception with **real extra behavior**: Spring applies exception translation (`PersistenceExceptionTranslationPostProcessor`) to `@Repository` beans, converting JDBC/JPA-specific exceptions into Spring's consistent unchecked `DataAccessException` family. Using the specific stereotype (`@Service`, `@Repository`, `@Controller`) over generic `@Component` is still best practice for **readability and intent** — it tells other developers exactly what role a class plays.

8. Multiple Implementations / Ambiguity [KEY]

```java
interface PaymentService {
    void pay();
}

@Component
class UpiPayment implements PaymentService {
    public void pay() { System.out.println("Paid via UPI"); }
}

@Component
class CardPayment implements PaymentService {
    public void pay() { System.out.println("Paid via Card"); }
}
```

If another class does `@Autowired private PaymentService paymentService;`, Spring finds **two** beans matching the `PaymentService` type and doesn't know which one to inject — this throws a `NoUniqueBeanDefinitionException` at startup.

@Primary

Marks one implementation as the default choice when multiple candidates exist.

```
@Component
@Primary
class UpiPayment implements PaymentService { }
```

@Qualifier

Explicitly names which bean to inject, overriding ambiguity at the injection point.

```
@Component("cardPayment")
class CardPayment implements PaymentService { }

@Component
class CheckoutService {
    private final PaymentService paymentService;

    @Autowired
    CheckoutService(@Qualifier("cardPayment") PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

@Primary vs @Qualifier: `@Primary` sets a *default* (used when no other hint is given); `@Qualifier` gives an *explicit* choice at each injection point and always wins over `@Primary` when both are present.

---

9. Bean Lifecycle [KEY]

```
Bean Definition (metadata read from annotations/config)
      ↓
Instantiation (constructor called)
      ↓
Dependency Injection (fields/setters populated)
      ↓
Initialization (@PostConstruct method runs, InitializingBean.afterPropertiesSet())
      ↓
Bean Ready (fully usable)
      ↓
Usage (application runs)
      ↓
Destruction (@PreDestroy method runs, on container shutdown)
```

```
@Component
public class DatabaseConnector {
```

```
    @PostConstruct
    public void init() {
        System.out.println("Connection pool initialized");
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("Connection pool closed");
    }
}
```

- **@PostConstruct** — runs once, right after dependency injection is complete; used for setup logic (opening connections, warming caches).
- **@PreDestroy** — runs once, just before the bean is removed from the container (e.g., on graceful application shutdown); used for cleanup (closing connections, releasing resources).

---

10. Layered Architecture [KEY]

```
Client (browser / Postman / frontend app)
  ↓  HTTP Request
Controller   → handles HTTP, request/response mapping, input validation
  ↓
Service      → business logic, orchestration, transactions
  ↓
Repository   → data access (talks to the database)
  ↓
Database
```

Responsibility of each layer: - **Controller:** Receives HTTP requests, delegates to the service layer, returns HTTP responses. Should contain *no* business logic. - **Service:** Contains the actual business rules, coordinates one or more repositories, manages transaction boundaries. - **Repository:** Talks directly to the database (via JDBC, Hibernate, or Spring Data JPA); contains no business logic, just data access.

Complete small example

```
// Repository
@Repository
class UserRepository {
    private final Map<Integer, String> db = new HashMap<>();
    String findById(int id) { return db.get(id); }
    void save(int id, String name) { db.put(id, name); }
}

// Service
@Service
class UserService {
    private final UserRepository userRepository;

    UserService(UserRepository userRepository) {
```

```
        this.userRepository = userRepository;
    }

    String getUserName(int id) {
        return userRepository.findById(id); // business logic could go here
    }
}

// Controller
@RestController
class UserController {
    private final UserService userService;

    UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/users/{id}")
    String getUser(@PathVariable int id) {
        return userService.getUserName(id);
    }
}
```

Why layers are useful: - **Separation of concerns** — each layer has one clear job, easier to reason about - **Testability** — you can unit-test the service layer with a mocked repository, without needing a real database or HTTP server - **Maintainability** — changing the database technology only touches the repository layer; changing business rules only touches the service layer - **Reusability** — the same service can be called from multiple controllers (REST, batch job, scheduled task)

Day 2 — Common Mistakes

- Putting business logic inside the Controller
- Injecting the Repository directly into the Controller, skipping the Service layer
- Using `@Autowired` on multiple beans of the same type without `@Primary`/`@Qualifier`, causing startup failure
- Forgetting that `@PostConstruct` runs *after* constructor injection, not during it — don't rely on injected fields inside the constructor body if they're set via field/setter injection

Day 2 — Interview Questions

Basic 1. **What does `@Component` do?** — Registers the class as a Spring-managed bean, discovered via component scanning. 2. **What is the default bean name for `@Component class PaymentService`?** — `paymentService` (class name, first letter lowercased).

Intermediate 3. **Difference between `@Component` and `@Bean`?** — `@Component` is a class-level annotation detected by scanning; `@Bean` is a method-level annotation inside `@Configuration`, used for beans you can't annotate directly (third-party classes). 4. **What extra behavior does `@Repository` add beyond `@Component`?** — Automatic exception translation into Spring's `DataAccessException` hierarchy.

Scenario-based 5. **You have two implementations of an interface and autowiring**

fails with `NoUniqueBeanDefinitionException`. How do you fix it? — Use `@Primary` on the default implementation, or `@Qualifier("beanName")` at the injection point.

Tricky 6. **Does `@Primary` override `@Qualifier`?** — No — `@Qualifier` is more specific and always wins when both are present at an injection point. 7. **When does `@PostConstruct` run relative to the constructor?** — After the constructor and after dependency injection is fully complete, not during construction.

Day 2 — Quick Revision

- `@Component` → generic bean; `@Service`/`@Controller` → semantic aliases; `@Repository` → adds exception translation
- `@Configuration` + @Bean → manual bean definition, used for third-party classes
- `@Autowired` → automatic dependency resolution by type
- Ambiguity fix: `@Primary` (default) or `@Qualifier` (explicit, wins over `@Primary`)
- Bean lifecycle: Definition → Instantiation → DI → `@PostConstruct` → Ready → `@PreDestroy`
- Layered architecture: Controller (HTTP) → Service (business logic) → Repository (data access) → DB
- Controller should never contain business logic; Repository should never contain business logic

DAY 3 — Spring Boot Introduction

1. What is Spring Boot? [KEY]

Spring Boot is a project built **on top of** Spring Framework that eliminates configuration overhead by providing auto-configuration, embedded servers, and opinionated starter dependencies — so you can build a production-ready Spring app with almost zero manual setup.

Spring vs Spring Boot

Aspect	Spring	Spring Boot
Configuration	Manual (XML or Java config)	Auto-configuration based on classpath
Server	External (deploy WAR to Tomcat)	Embedded server (Tomcat/Jetty) built in
Setup time	High — wire everything yourself	Low — starters + auto-config
Dependency management	Manual version management	Starters bundle compatible versions
Production features	Build yourself	Built-in (Actuator: health checks, metrics)

Problems with traditional Spring

- Verbose XML/Java configuration for even simple apps
- Manually choosing compatible versions of dozens of libraries
- Manually configuring and deploying an external servlet container
- No built-in production monitoring

Spring Boot's solutions

- **Auto-configuration:** Spring Boot inspects what's on the classpath (e.g., is `spring-webmvc` present?) and automatically configures sensible defaults — no manual bean wiring needed for common setups.
- **Starter dependencies:** Curated dependency bundles (e.g., `spring-boot-starter-web`) that pull in everything needed for a use case, with tested-compatible versions.
- **Embedded server:** Tomcat (or Jetty/Undertow) is bundled inside the JAR — `java -jar app.jar` runs a fully working web server, no external deployment needed.
- **Production-ready features:** Spring Boot Actuator exposes health checks, metrics, and monitoring endpoints out of the box.

---

2. Creating a Spring Boot Project

Spring Initializr (start.spring.io) generates a ready-to-run project skeleton — you pick build tool, language, Spring Boot version, and starter dependencies.

Project structure

```
my-app/
├── pom.xml
├── src/main/java/com/example/app/
│   └── MyAppApplication.java     ← main class
├── src/main/resources/
│   └── application.properties
└── src/test/java/...
```

Main class

```java
@SpringBootApplication
public class MyAppApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyAppApplication.class, args);
    }
}
```

What `@SpringBootApplication` really is [KEY]

It's a **meta-annotation** combining three annotations:

```
@SpringBootApplication
   = @SpringBootConfiguration   (a specialized @Configuration)
   + @EnableAutoConfiguration    (triggers classpath-based auto-config)
   + @ComponentScan              (scans the current package + sub-packages)
```

This single annotation is why a Spring Boot app needs almost no manual setup — it configures itself, scans for beans, and enables Spring MVC/JPA/etc. automatically based on what's on the classpath.

---

3. Spring Boot Starters [KEY]

Starters are dependency descriptors that bundle everything needed for a feature, so you add one line to `pom.xml` instead of tracking a dozen individual libraries.

Starter	Brings in
`spring-boot-starter-web`	Spring MVC, embedded Tomcat, Jackson (JSON) — for building REST APIs
`spring-boot-starter-data-jpa`	Spring Data JPA, Hibernate, JDBC support
`spring-boot-starter-security`	Spring Security — authentication/authorization
`spring-boot-starter-test`	JUnit, Mockito, Spring Test — testing tools
`spring-boot-starter-validation`	Bean Validation (Hibernate Validator) — `@NotNull`, `@Size`, etc.

Why starters are useful: They remove version-compatibility guesswork — all libraries in a starter are tested to work together for that Spring Boot release.

---

4. application.properties

Central place for externalized configuration — no code changes needed to alter behavior across environments.

```
server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/testdb
spring.datasource.username=root
spring.datasource.password=password

logging.level.org.springframework=INFO

# custom property
app.feature.enabled=true
```

- `server.port` — port the embedded server listens on
- `spring.datasource.*` — database connection details
- `logging.level.*` — controls log verbosity per package
- Custom properties can be injected into beans with `@Value("${app.feature.enabled}")`

---

5. CommandLineRunner

An interface for running code **once, right after** the application context is fully loaded and the app has started — commonly used for startup tasks (seeding data, one-off jobs).

```java
@Component
public class DataSeeder implements CommandLineRunner {
    @Override
    public void run(String... args) {
        System.out.println("App started — raw args: " + Arrays.toString(args));
    }
}
```

---

6. ApplicationRunner

Same purpose as `CommandLineRunner`, but gives you a parsed `ApplicationArguments` object instead of a raw `String[]` — useful when you need structured access to option-style arguments (`--key=value`).

```java
@Component
public class StartupTask implements ApplicationRunner {
    @Override
    public void run(ApplicationArguments args) {
        System.out.println("Option names: " + args.getOptionNames());
    }
}
```

CommandLineRunner vs ApplicationRunner

CommandLineRunner	ApplicationRunner
Receives raw `String... args`	Receives parsed `ApplicationArguments`
Simpler for basic use	Better for parsing `--option=value` style arguments
`run(String... args)`	`run(ApplicationArguments args)`
Both run once, right after context startup, in the order defined by `@Order` if multiple exist	Same

---

Day 3 — Common Mistakes

- Placing the main class in a sub-package instead of the root package — component scan then misses sibling packages
- Hardcoding config values instead of using `application.properties`
- Confusing Spring Boot's auto-configuration with "magic" — it's driven entirely by what's on the classpath + conditional annotations (`@ConditionalOnClass`, etc.), not randomness
- Forgetting starters bundle transitive dependencies — adding both `starter-web` and manually adding Tomcat again causes conflicts

Day 3 — Interview Questions (20)

Basic 1. **What is Spring Boot?** — A framework on top of Spring that provides auto-configuration, embedded servers, and starter dependencies for rapid, low-config application development. 2. **What does `@SpringBootApplication` combine?** — `@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan`. 3. **What is an embedded server?** — A servlet container (like Tomcat) bundled directly inside the application JAR, so the app is self-contained and runnable with `java -jar`. 4. **What file holds Spring Boot configuration?** — `application.properties` or `application.yml`. 5. **What does `spring-boot-starter-web` include?** — Spring MVC, embedded Tomcat, and Jackson for JSON.

Intermediate 6. **How does auto-configuration decide what to configure?** — By inspecting the classpath and existing bean definitions using conditional annotations like `@ConditionalOnClass` and `@ConditionalOnMissingBean`. 7. **Can you override an auto-configured bean?** — Yes — define your own bean of the same type; Spring Boot's conditional annotations back off when a user-defined bean already exists. 8. **Difference between Spring and Spring Boot?** — Spring requires manual configuration and external servers; Spring Boot auto-configures and embeds a server. 9. **What is Spring Initializr?** — A web tool (start.spring.io) to generate a preconfigured Spring Boot project skeleton. 10. **How do you change the default port?** — `server.port=<port>` in `application.properties`. 11. **What is `spring-boot-starter-test` used for?** — Bundles JUnit, Mockito, and Spring Test utilities for testing. 12. **Difference between `CommandLineRunner` and `ApplicationRunner`?** — See comparison table above; core difference is raw `String[]` vs parsed `ApplicationArguments`. 13. **Can you disable a specific auto-configuration?** — Yes, using `@SpringBootApplication(exclude = SomeAutoConfiguration.class)`. 14. **What is `@ConfigurationProperties` used for?** — Binding a group of related properties (e.g., `app.mail.*`) into a strongly-typed POJO instead of individual `@Value` injections. 15. **How does Spring Boot know to configure a `DataSource`?** — It detects a JDBC driver + `spring.datasource.*` properties on the classpath/config and auto-configures a `DataSource` bean.

Scenario-based 16. **Your app needs different DB settings for dev and production. How do you manage that in Spring Boot?** — Use Spring Profiles (`application-`

dev.properties, application-prod.properties) and activate one via spring.profiles.active. 17. **You added spring-boot-starter-data-jpa but the app fails to start with "Cannot determine embedded database driver." What's happening?** — Spring Boot tried to auto-configure a database connection but found no datasource URL/driver configured — either add DB connection properties or an in-memory DB dependency (H2).

Tricky 18. **Is @SpringBootApplication mandatory?** — No — you could manually apply @Configuration, @EnableAutoConfiguration, and @ComponentScan separately; @SpringBootApplication is just a convenience. 19. **Does Spring Boot replace Spring Framework?** — No — it's built *on top of* Spring Framework; all core Spring concepts (IoC, DI, AOP) still apply underneath. 20. **What happens if two starters bring conflicting versions of the same library?** — Spring Boot's dependency management (via the parent BOM) resolves it to a tested, compatible version, unless explicitly overridden.

Day 3 — Quick Revision

- Spring Boot = Spring + auto-configuration + embedded server + starters
- @SpringBootApplication = @SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan
- Starters bundle compatible, tested dependency sets for a feature area
- application.properties/.yml externalizes configuration
- CommandLineRunner/ApplicationRunner run once at startup, after context is ready
- Auto-configuration is classpath-driven and conditional, not "magic"
- Embedded Tomcat means java -jar app.jar is enough to run a full web server

DAY 4 — First REST API

1. REST API — Basics

- **API (Application Programming Interface):** A contract that lets two software systems communicate.
- **Web API:** An API exposed over HTTP, usable by any client (browser, mobile app, another server).
- **REST (REpresentational State Transfer):** An architectural style for designing web APIs around **resources**, using standard HTTP methods and stateless communication.
- **REST principles:** Statelessness (server stores no client session state between requests), resource-based URLs, uniform interface (standard HTTP verbs), representation of resources as JSON/XML.
- **Resource:** A "thing" the API exposes — e.g., a `User`, an `Order`.
- **Endpoint:** A specific URL representing an operation on a resource, e.g., `GET /api/users`.
- **JSON (JavaScript Object Notation):** The standard lightweight data format REST APIs use to represent resources.

---

2. HTTP Methods [KEY]

Method	Purpose	Body?	Idempotent?
GET	Retrieve a resource	No	Yes
POST	Create a new resource	Yes	No
PUT	Fully replace/update a resource	Yes	Yes
PATCH	Partially update a resource	Yes	No (usually)
DELETE	Remove a resource	No (usually)	Yes

Idempotent means calling it multiple times has the same effect as calling it once (e.g., deleting the same resource twice still leaves it deleted — no extra side effect).

---

3. HTTP Status Codes [KEY]

2xx — Success

- **200 OK** — Request succeeded (typical for GET, PUT)
- **201 Created** — A new resource was successfully created (typical for POST)
- **204 No Content** — Success, but no response body (typical for DELETE)

4xx — Client Error

- **400 Bad Request** — Malformed request (invalid JSON, missing required field)
- **401 Unauthorized** — Missing/invalid authentication
- **403 Forbidden** — Authenticated, but not allowed to perform this action
- **404 Not Found** — Resource doesn't exist
- **409 Conflict** — Request conflicts with current state (e.g., duplicate unique field)

5xx — Server Error

- **500 Internal Server Error** — Unhandled exception on the server
- **502 Bad Gateway** — An upstream server (behind a proxy) returned an invalid response
- **503 Service Unavailable** — Server temporarily overloaded or down for maintenance

---

4. First REST API — Example

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    private final List<String> users = new ArrayList<>(List.of("Alice", "Bob"));

    @GetMapping
    public List<String> getAllUsers() {
        return users;
    }

    @GetMapping("/{id}")
    public String getUserById(@PathVariable int id) {
        return users.get(id);
    }
}
```

Requests:

```
GET /api/users        → ["Alice", "Bob"]
GET /api/users/1      → "Bob"
```

---

5. Returning JSON

When a controller method returns a Java object, Spring Boot uses **Jackson** (bundled with `spring-boot-starter-web`) to automatically convert it:

- **Serialization:** Java object → JSON (for responses)
- **Deserialization:** JSON → Java object (for request bodies, via `@RequestBody`)

```
public class User {
    private int id;
    private String name;
    // getters/setters
}

@GetMapping("/{id}")
public User getUser(@PathVariable int id) {
    return new User(id, "Alice"); // Jackson converts this to {"id":1,"name":"Alice"}
}
```

This automatic conversion happens because `@RestController` implies `@ResponseBody` on every method, which tells Spring to write the return value directly into the HTTP response body (via a message converter — Jackson) rather than resolving it as a view name.

---

6. @RequestMapping and HTTP-Specific Annotations [KEY]

```java
@RestController
@RequestMapping("/api/users")   // class-level: base path for all methods below
public class UserController {

    @GetMapping             // shorthand for @RequestMapping(method = GET)
    public List<User> getAll() { ... }

    @PostMapping
    public User create(@RequestBody User user) { ... }

    @PutMapping("/{id}")
    public User update(@PathVariable int id, @RequestBody User user) { ... }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable int id) { ... }

    @PatchMapping("/{id}")
    public User partialUpdate(@PathVariable int id, @RequestBody Map<String, Object> updat
}
```

@GetMapping, @PostMapping, etc. are just shorthand for @RequestMapping(method = RequestMethod.GET) and so on — introduced for readability.

---

7. Internal Request Flow [KEY] (very common interview topic)

```
Client (browser/Postman)
 ↓  sends HTTP Request
Embedded Tomcat            → receives the raw HTTP request (socket-level)
 ↓
DispatcherServlet          → Spring MVC's "front controller" — single entry point for all
 ↓
Handler Mapping            → determines WHICH controller method matches the URL + HTTP met
 ↓
Controller                 → executes, delegates to Service
 ↓
Service                    → business logic
 ↓
Repository                 → talks to the database
 ↓
Database
 ↓  data flows back up
Response object returned by Controller
 ↓
Jackson                    → serializes the Java object into JSON
 ↓
DispatcherServlet writes the JSON into the HTTP response
```

```
 ↓
Client receives JSON response
```

Key components explained: - **Embedded Tomcat:** The actual HTTP server that accepts TCP connections and parses raw HTTP requests. - **DispatcherServlet:** The single servlet Spring MVC registers to intercept *all* incoming requests; it orchestrates the entire request-handling pipeline. - **HandlerMapping:** Matches the incoming URL + HTTP method to the correct `@RequestMapping`-annotated controller method. - **Jackson (HttpMessageConverter):** Converts between Java objects and JSON automatically for both requests (`@RequestBody`) and responses.

Day 4 — Common Mistakes

- Forgetting `@RestController` (using plain `@Controller`) and then wondering why a JSON string is treated as a view name
- Mismatching HTTP methods for actions (e.g., using GET to delete a resource — breaks idempotency and REST conventions, and is dangerous since GET requests can be triggered by prefetching/crawlers)
- Not returning proper status codes (e.g., always returning 200 even for errors)
- Confusing "REST" with "just any JSON API" — true REST implies statelessness, resource orientation, and correct use of HTTP semantics

Day 4 — Interview Questions

Basic 1. **What is REST?** — An architectural style for APIs based on stateless, resource-oriented communication using standard HTTP methods. 2. **What does `@RestController` do?** — Combines `@Controller` + `@ResponseBody`, so return values are written directly as the response body (typically JSON) instead of resolved as a view name. 3. **Difference between 200 and 201?** — 200 = generic success; 201 = a new resource was successfully created.

Intermediate 4. **What is DispatcherServlet and why is it called the "front controller"?** — It's the single entry point that intercepts every incoming request and routes it to the correct controller method — a design pattern called Front Controller. 5. **How does Spring know which Java object maps to incoming JSON?** — Via Jackson (`HttpMessageConverter`), which deserializes the JSON request body into the type declared on the `@RequestBody` parameter.

Scenario-based 6. **A client says your DELETE endpoint returns a body but they expected nothing.** — Return `ResponseEntity<Void>` with status `204 No Content` rather than any body.

Tricky 7. **Is GET allowed to change server state?** — No, by REST convention GET must be safe/read-only; using it to mutate state breaks idempotency guarantees and can cause bugs with caching/prefetching.

Day 4 — Quick Revision

- REST = resource-based, stateless API design using HTTP verbs
- GET/PUT/DELETE = idempotent; POST/PATCH = generally not
- 2xx = success, 4xx = client error, 5xx = server error
- `@RestController` = `@Controller` + `@ResponseBody`
- Jackson handles JSON ⇄ Java object conversion automatically
- Request flow: Tomcat → DispatcherServlet → HandlerMapping → Controller → Service → Repository → DB → Jackson → JSON response

DAY 5 — POST, PUT, DELETE

POST — Creating Resources

@RequestBody tells Spring to deserialize the incoming JSON request body into a Java object.

```
@PostMapping
public ResponseEntity<User> createUser(@RequestBody User user) {
    User saved = userService.save(user);
    return ResponseEntity.status(HttpStatus.CREATED).body(saved);
}
```

DTO (Data Transfer Object) concept [KEY]

A DTO is a plain object used purely to transfer data between layers (typically between client and controller), **separate from your database entity**.

Why DTOs matter: - Prevents exposing internal database structure (entity fields like password hashes) directly to clients - Lets you shape the API contract independently from the database schema - Allows different validation rules for create vs update operations

```
public class UserRequestDto {
    @NotBlank
    private String name;
    @Email
    private String email;
    // no "id" field — client shouldn't set the ID on creation
}
```

PUT — Updating Resources

```
@PutMapping("/{id}")
public ResponseEntity<User> updateUser(@PathVariable int id, @RequestBody User user) {
    User updated = userService.update(id, user);
    return ResponseEntity.ok(updated);
}
```

Full update vs partial update

- **PUT** conventionally replaces the **entire** resource — any field not included in the request is expected to be cleared/reset.
- **PATCH** updates only the fields provided — the rest remain untouched.

PUT vs PATCH

Aspect	PUT	PATCH
Update type	Full replacement	Partial update
Idempotent	Yes	Not guaranteed
Missing fields	Treated as null/cleared	Left unchanged
Typical body	Complete resource representation	Only the fields being changed

---

DELETE — Deleting Resources

```java
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteUser(@PathVariable int id) {
    userService.delete(id);
    return ResponseEntity.noContent().build(); // 204
}
```

Proper status codes: `204 No Content` on success (nothing to return), `404 Not Found` if the resource doesn't exist.

ResponseEntity [KEY]

ResponseEntity<T> wraps the response body **together with** the HTTP status code and headers, giving full control over the HTTP response — unlike returning a plain object (which always defaults to 200 OK).

```java
return ResponseEntity.ok(user);                                  // 200 + body
return ResponseEntity.status(HttpStatus.CREATED).body(user); // 201 + body
return ResponseEntity.noContent().build();                       // 204, no body
return ResponseEntity.notFound().build();                        // 404, no body
```

Why it's useful: REST conventions require different status codes for different outcomes (create → 201, delete → 204, not found → 404). Returning a plain POJO always yields `200 OK`, which is often semantically wrong. ResponseEntity lets you express the true outcome of the request.

Complete CRUD REST API (in-memory)

```java
@RestController
@RequestMapping("/api/users")
public class UserController {

    private final Map<Integer, User> db = new HashMap<>();
    private int nextId = 1;

    @GetMapping
    public List<User> getAll() {
        return new ArrayList<>(db.values());
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getById(@PathVariable int id) {
        User user = db.get(id);
        return (user != null) ? ResponseEntity.ok(user) : ResponseEntity.notFound().build(
    }

    @PostMapping
```

```
    public ResponseEntity<User> create(@RequestBody User user) {
        user.setId(nextId++);
        db.put(user.getId(), user);
        return ResponseEntity.status(HttpStatus.CREATED).body(user);
    }

    @PutMapping("/{id}")
    public ResponseEntity<User> update(@PathVariable int id, @RequestBody User user) {
        if (!db.containsKey(id)) return ResponseEntity.notFound().build();
        user.setId(id);
        db.put(id, user);
        return ResponseEntity.ok(user);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable int id) {
        if (!db.containsKey(id)) return ResponseEntity.notFound().build();
        db.remove(id);
        return ResponseEntity.noContent().build();
    }
}
```

Code explanation: - `db` and `nextId` simulate a database using an in-memory map — this is replaced by a real repository from Day 11 onward. - `getById` checks existence before responding, returning `404` instead of `null`/`200` when the resource is missing — this is a very common interview scenario. - `create` assigns the ID server-side (never trust a client-supplied ID for creation) and responds `201`. - `update`/`delete` both check existence first, so operating on a non-existent ID returns `404` rather than silently succeeding.

Day 5 — Common Mistakes

- Letting the client set the resource ID on POST (IDs should be server-generated)
- Returning `200 OK` for every response regardless of outcome
- Not checking whether a resource exists before updating/deleting (silently creating a new record instead of erroring)
- Confusing PUT (idempotent, full replace) with PATCH (partial, not guaranteed idempotent)

Day 5 — Interview Questions

Basic 1. **What annotation reads the JSON body into a Java object?** — `@RequestBody`. 2. **What status code should a successful DELETE return?** — `204 No Content` (or `200 OK` if returning the deleted entity).

Intermediate 3. **Why use a DTO instead of returning the entity directly?** — Decouples the API contract from the database schema, and avoids leaking sensitive/internal fields. 4. **Why is `ResponseEntity` preferred over returning a plain object?** — It gives explicit control over status code and headers, not just the body.

Scenario-based 5. **A client sends a PUT request but omits a field that had a value before. What should happen?** — Under PUT semantics, that field should be cleared/reset (full replacement) — if partial update is desired, PATCH should be used instead.

Tricky 6. **Is PATCH guaranteed to be idempotent?** — Not by specification — it depends on implementation (e.g., "increment counter by 1" via PATCH would not be idempotent), unlike PUT which is expected to be idempotent by convention.

Day 5 — Quick Revision

- POST → create (201), PUT → full update (200), PATCH → partial update (200), DELETE → remove (204)
- `@RequestBody` deserializes JSON → Java object
- DTOs separate API contract from database entity
- `ResponseEntity` gives full control over status code + body + headers
- Always validate resource existence before update/delete → return 404 if missing

DAY 6 — Path Variables & Query Parameters

@PathVariable [KEY]

Extracts a value embedded directly in the URL path — used to identify a **specific resource**.

```
GET /users/10
```

```java
@GetMapping("/users/{id}")
public User getUser(@PathVariable int id) {
    return userService.findById(id);
}
```

If the variable name in the URL differs from the method parameter name, bind explicitly:

```java
@GetMapping("/users/{userId}")
public User getUser(@PathVariable("userId") int id) { ... }
```

@RequestParam [KEY]

Extracts values from the **query string** — typically used for filtering, searching, sorting, or pagination (optional data, not resource identity).

```
GET /users?name=Rachit&age=25
```

```java
@GetMapping("/users")
public List<User> searchUsers(
        @RequestParam String name,
        @RequestParam(required = false) Integer age) {
    ...
}
```

- **Required parameter (default):** `@RequestParam String name` — request fails with `400 Bad Request` if `name` is missing.
- **Optional parameter:** `@RequestParam(required = false) Integer age` — allows the parameter to be absent (value will be `null`).
- **Default value:** `@RequestParam(defaultValue = "10") int limit` — used when the parameter is missing.
- **Multiple parameters:** simply declare multiple `@RequestParam` arguments; each maps to its own query key.

@PathVariable vs @RequestParam

Aspect	@PathVariable	@RequestParam
Location	Part of the URL path	Query string (after ?)
Purpose	Identifies a specific resource	Filters, sorts, or configures a request
Example	`/users/10`	`/users?age=25`
Optionality	Usually mandatory (part of the route)	Often optional

---

@RequestHeader

Extracts a value from an HTTP header — commonly used for authentication tokens, content negotiation, or client metadata.

```
@GetMapping("/profile")
public User getProfile(@RequestHeader("Authorization") String token) {
    // e.g., token = "Bearer eyJhbGciOi..."
    String jwt = token.replace("Bearer ", "");
    return userService.getUserFromToken(jwt);
}
```

Real-world usage with JWT: In secured APIs, the client sends a JWT in the `Authorization` header on every request; the server extracts it via `@RequestHeader`, validates the signature/expiry, and identifies the authenticated user — this is the foundation of stateless, token-based authentication (commonly paired with Spring Security filters rather than manual extraction in each controller, but the concept is the same).

Combining Parameters — Full Example

```
@PutMapping("/users/{id}/orders")
public ResponseEntity<Order> updateOrder(
        @RequestHeader("Authorization") String token,
        @PathVariable int id,
        @RequestParam(defaultValue = "false") boolean notify,
        @RequestBody OrderUpdateDto orderUpdate) {

    validateToken(token);
    Order updated = orderService.update(id, orderUpdate, notify);
    return ResponseEntity.ok(updated);
}
```

Request processing order (conceptually): 1. `DispatcherServlet` routes the request to this method based on URL pattern + HTTP method. 2. Spring extracts `id` from the path (`@PathVariable`). 3. Spring reads `notify` from the query string, applying the default if absent (`@RequestParam`). 4. Spring reads the `Authorization` header (`@RequestHeader`). 5. Jackson deserializes the JSON body into `OrderUpdateDto` (`@RequestBody`). 6. The fully-populated arguments are passed to the controller method.

Day 6 — Common Mistakes

- Using `@RequestParam` to identify a specific resource (should be `@PathVariable`) — hurts REST clarity (`/users?id=5` instead of `/users/5`)
- Forgetting `required = false` and getting unexpected `400` errors for genuinely optional filters
- Manually parsing `Authorization` headers everywhere instead of centralizing token validation (in a filter/interceptor) once security is introduced
- Mismatching path variable names between the URL template and the parameter (forgetting the explicit `@PathVariable("name")` binding)

Day 6 — Interview Questions

Basic 1. **What does `@PathVariable` extract?** — A value embedded in the URL path itself. 2. **What does `@RequestParam` extract?** — A value from the query string.

Intermediate 3. **How do you make a `@RequestParam` optional with a default?** — `@RequestParam(required = false, defaultValue = "10")`. 4. **When would you use `@RequestHeader`?** — To read metadata like auth tokens, content type, or client info sent in HTTP headers.

Scenario-based 5. **You need an endpoint to search users by an optional name and optional minimum age.** — Use two `@RequestParam(required = false)` arguments, applying filtering logic conditionally based on which are present.

Tricky 6. **Can a query parameter and path variable share the same name?** — Yes, technically, but it's confusing — Spring binds each to its own annotated parameter independently; avoid the naming collision in practice.

Day 6 — Quick Revision

- `@PathVariable` → identifies a resource (URL path segment)
- `@RequestParam` → filters/configures a request (query string)
- `@RequestHeader` → reads HTTP headers (commonly auth tokens)
- Use `required = false` / `defaultValue` for optional query params
- All four (`@PathVariable`, `@RequestParam`, `@RequestHeader`, `@RequestBody`) can be combined in a single method signature

DAY 7 — Exception Handling & Logging

1. Why Exception Handling?

Bad API response (unhandled exception):

```
{
  "timestamp": "2026-08-13T10:00:00",
  "status": 500,
  "error": "Internal Server Error",
  "trace": "java.lang.NullPointerException: ... [50 lines of stack trace] ..."
}
```

This leaks internal implementation details, is unhelpful to the client, and always returns a generic 500 even for something as simple as “user not found.”

Proper API response:

```
{
  "timestamp": "2026-08-13T10:00:00",
  "status": 404,
  "message": "User not found with id: 10",
  "path": "/api/users/10"
}
```

Clean, predictable, correct status code, no internal leakage.

---

2. Service Layer Delegation

The Controller should never contain business rules or manual exception construction scattered everywhere — it delegates to the Service, which throws meaningful exceptions when something goes wrong.

```
@Service
public class UserService {
    private final UserRepository userRepository;

    public User findById(int id) {
        return userRepository.findById(id)
                .orElseThrow(() -> new UserNotFoundException("User not found with id: " +
    }
}
```

---

3. @ExceptionHandler

Handles exceptions **local to a single controller** — good for controller-specific error handling, but doesn’t scale across an app with many controllers.

```
@RestController
public class UserController {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handleNotFound(UserNotFoundException ex) {
```

```
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
    }
}
```

---

4. @RestControllerAdvice [KEY]

Handles exceptions **globally, across all controllers** — the recommended, centralized approach.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleUserNotFound(UserNotFoundException ex, Http
        ErrorResponse error = new ErrorResponse(
                LocalDateTime.now(), 404, ex.getMessage(), request.getRequestURI());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex, HttpServletRequest re
        ErrorResponse error = new ErrorResponse(
                LocalDateTime.now(), 500, "Something went wrong", request.getRequestURI())
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
    }
}
```

`@RestControllerAdvice` = `@ControllerAdvice` + `@ResponseBody` — it intercepts exceptions thrown from *any* `@RestController` in the app and routes them to the matching `@ExceptionHandler` method here, keeping error-handling logic in one place.

@ExceptionHandler vs @RestControllerAdvice

Aspect	@ExceptionHandler (alone)	@RestControllerAdvice
Scope	Only the controller it's defined in	Global — all controllers
Reusability	Duplicated per controller	Centralized, written once
Best practice	Small apps / very specific cases	Standard for real applications

---

5. Custom Exceptions

```
public class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(String message) {
        super(message);
    }
}
```

Used in the service:

```
throw new UserNotFoundException("User not found with id: " + id);
```

Why custom exceptions? They express *intent* clearly (a `UserNotFoundException` is far more meaningful than a generic `RuntimeException`), and let `@RestControllerAdvice` map different exception types to different HTTP status codes precisely.

---

6. Standard Error Response

```
public class ErrorResponse {
    private LocalDateTime timestamp;
    private int status;
    private String message;
    private String path;
    // constructor, getters
}
```

Produces:

```
{
  "timestamp": "2026-08-13T10:00:00",
  "status": 404,
  "message": "User not found with id: 10",
  "path": "/api/users/10"
}
```

A consistent error shape means **every client-facing failure looks the same**, making the API predictable and easy to consume.

---

7. Logging [KEY]

Why logging is needed

Logs are the primary way to understand what happened in a running application — especially in production, where you cannot attach a debugger.

System.out.println vs Logger

Aspect	System.out.println	Logger (SLF4J)
Configurability	None — always prints	Configurable levels, output destinations (file, console, remote)
Performance	Always executes, even in production	Can be filtered by level, avoiding unnecessary work
Structure	Plain text only	Supports levels, timestamps, class context, structured/JSON logging
Production use	Not recommended	Standard practice

Log Levels (lowest to highest severity)

- **DEBUG** — detailed diagnostic info, useful during development
- **INFO** — high-level application events (e.g., "User created successfully")
- **WARN** — something unexpected happened but the app can continue
- **ERROR** — a failure occurred that needs attention

```
@Service
public class UserService {
    private static final Logger log = LoggerFactory.getLogger(UserService.class);

    public User findById(int id) {
        log.info("Fetching user with id={}", id);
        return userRepository.findById(id)
                .orElseThrow(() -> {
                    log.warn("User not found for id={}", id);
                    return new UserNotFoundException("User not found with id: " + id);
                });
    }
}
```

Best practices: - Never log sensitive data (passwords, tokens, full card numbers) - Use parameterized logging (`log.info("id={}", id)`) instead of string concatenation — avoids unnecessary string building when the log level is disabled - Log at ERROR only for genuine failures, not expected business conditions (e.g., "user not found" is often just INFO/WARN, not ERROR)

Day 7 — Common Mistakes

- Letting exceptions propagate as raw `500` errors with stack traces exposed to the client
- Duplicating `@ExceptionHandler` methods across every controller instead of centralizing with `@RestControllerAdvice`
- Using `System.out.println` for debugging in production code
- Logging entire request/response bodies that may contain sensitive user data

Day 7 — Interview Questions

Basic 1. **What does `@RestControllerAdvice` do?** — Provides global, centralized exception handling across all `@RestController` classes. 2. **What are the standard log levels?** — DEBUG, INFO, WARN, ERROR (from least to most severe).

Intermediate 3. **Difference between `@ExceptionHandler` and `@RestControllerAdvice`?** — `@ExceptionHandler` alone is scoped to one controller; `@RestControllerAdvice` applies globally. 4. **Why create custom exceptions instead of using generic `RuntimeException`?** — Clearer intent, and lets the global handler map specific exception types to specific HTTP statuses.

Scenario-based 5. **Your API currently returns a raw stack trace to clients on error. How do you fix this?** — Add a `@RestControllerAdvice` class with handlers that catch exceptions and return a clean, standardized `ErrorResponse` with the correct status code.

Tricky 6. **If you have both a controller-local `@ExceptionHandler` and a global one for the same exception type, which wins?** — The controller-local one takes precedence for requests handled by that specific controller.

Day 7 — Quick Revision

- Never expose raw stack traces to API clients
- `@ExceptionHandler` = local; `@RestControllerAdvice` = global (preferred)
- Custom exceptions communicate intent and map cleanly to HTTP status codes
- Standard error response = timestamp + status + message + path
- Logger > `System.out.println` — configurable, leveled, production-safe
- Log levels: DEBUG < INFO < WARN < ERROR

DAY 8 — JDBC Fundamentals

1. Why JDBC?

Java applications don't speak "MySQL" natively — they need a standard bridge to send SQL and receive results regardless of which database vendor is used. **JDBC (Java Database Connectivity)** is that standard API: write code once against the JDBC interfaces, and swap the underlying database driver without rewriting application logic.

2. JDBC Architecture

```
Java Application
      ↓
JDBC API (java.sql.*)        ← standard interfaces
      ↓
JDBC Driver (vendor-specific, e.g., mysql-connector-j)
      ↓
MySQL (or any other RDBMS)
```

The JDBC API defines *interfaces* (`Connection`, `Statement`, `ResultSet`); each database vendor ships a **driver** — a concrete implementation of those interfaces that knows how to talk to that specific database over its native protocol.

3. Important JDBC Classes/Interfaces [KEY]

Class/Interface	Role
`Driver`	Vendor-specific implementation that establishes the actual connection
`DriverManager`	Factory that locates the correct `Driver` and produces a `Connection`
`Connection`	Represents an open session with the database
`Statement`	Executes static SQL (no parameters)
`PreparedStatement`	Executes precompiled, parameterized SQL — safer and faster (Day 9)
`ResultSet`	A cursor over the rows returned by a SELECT query

4. Database Connection — Complete Example

```java
import java.sql.*;

public class JdbcExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/testdb";
        String user = "root";
        String password = "password";

        try (Connection con = DriverManager.getConnection(url, user, password)) {
            System.out.println("Connected successfully!");
```

```
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Line-by-line: - `url` — JDBC connection string: protocol (`jdbc:mysql`), host, port, database name. - `DriverManager.getConnection(...)` — asks the driver manager to find a matching driver and open a `Connection`. - `try (...)` — try-with-resources ensures the connection is closed automatically.

5. try-with-resources [KEY]

Any resource implementing `AutoCloseable` (Connection, Statement, ResultSet all do) can be declared inside `try (...)`, and Java **guarantees it's closed automatically** when the block exits — even if an exception is thrown.

```
try (Connection con = DriverManager.getConnection(url, user, password);
     PreparedStatement ps = con.prepareStatement("SELECT * FROM users");
     ResultSet rs = ps.executeQuery()) {

    while (rs.next()) {
        System.out.println(rs.getString("name"));
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```

Why it matters: Manually closing each resource in a `finally` block is verbose and error-prone (forgetting to close a `Connection` leaks database connections, eventually exhausting the connection pool). try-with-resources removes this entire class of bugs.

6. CRUD Using JDBC

INSERT

```
String sql = "INSERT INTO users (name, email) VALUES (?, ?)";
try (Connection con = DriverManager.getConnection(url, user, password);
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, "Alice");
    ps.setString(2, "alice@example.com");
    int rows = ps.executeUpdate(); // returns number of affected rows
    System.out.println(rows + " row(s) inserted");
}
```

SELECT

```
String sql = "SELECT id, name, email FROM users WHERE id = ?";
try (Connection con = DriverManager.getConnection(url, user, password);
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setInt(1, 1);
    try (ResultSet rs = ps.executeQuery()) {
```

```
        if (rs.next()) {
            System.out.println(rs.getInt("id") + " " + rs.getString("name"));
        }
    }
}
```

UPDATE

```
String sql = "UPDATE users SET email = ? WHERE id = ?";
try (Connection con = DriverManager.getConnection(url, user, password);
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, "new@example.com");
    ps.setInt(2, 1);
    ps.executeUpdate();
}
```

DELETE

```
String sql = "DELETE FROM users WHERE id = ?";
try (Connection con = DriverManager.getConnection(url, user, password);
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setInt(1, 1);
    ps.executeUpdate();
}
```

Key line explanations: - `executeUpdate()` — used for INSERT/UPDATE/DELETE; returns the number of rows affected. - `executeQuery()` — used for SELECT; returns a `ResultSet`. - `rs.next()` — advances the cursor to the next row; returns `false` when there are no more rows. - `ps.setX(index, value)` — binds a value to the `?` placeholder at position `index` (1-based).

Day 8 — Common Mistakes

- Not closing `Connection`/`Statement`/`ResultSet` (connection leaks) — solved by try-with-resources
- Concatenating SQL strings with user input directly instead of using `PreparedStatement` (SQL injection risk — covered Day 9)
- Forgetting `executeUpdate()` vs `executeQuery()` — using the wrong one throws an exception
- Opening a new `Connection` per query instead of reusing/pooling connections in real applications

Day 8 — Interview Questions

Basic 1. **What is JDBC?** — A standard Java API for connecting to and interacting with relational databases. 2. **What does `DriverManager.getConnection()` return?** — A `Connection` object representing a live database session.

Intermediate 3. **Why use try-with-resources with JDBC?** — Guarantees automatic closing of `Connection`/`Statement`/`ResultSet`, preventing resource leaks even when exceptions occur.

4. **What does `executeUpdate()` return, and when is it used?** — The number of affected rows; used for INSERT/UPDATE/DELETE.

Scenario-based 5. **Your app is slowly running out of database connections in production. What's a likely JDBC-related cause?** — Connections not being closed properly — missing try-with-resources or `finally` blocks, causing leaks.

Tricky 6. **Can you reuse a `ResultSet` after its parent `Connection` is closed?** — No — closing the `Connection` closes all resources derived from it.

Day 8 — Quick Revision

- JDBC = standard API bridging Java and relational databases via vendor-specific drivers
- Key classes: `DriverManager`, `Connection`, `Statement`/`PreparedStatement`, `ResultSet`
- try-with-resources auto-closes JDBC resources, preventing leaks
- `executeQuery()` → SELECT (returns `ResultSet`); `executeUpdate()` → INSERT/UPDATE/DELETE (returns row count)
- `rs.next()` advances the cursor; loop until it returns `false`

DAY 9 — PreparedStatements & Transactions

PreparedStatement [KEY]

What is it?

A precompiled SQL statement with placeholders (?) for parameters, which are bound safely at execution time rather than being concatenated into the SQL string.

Why use it?

- **Prevents SQL Injection** — parameter values are never interpreted as SQL code, only as data.
- **Performance** — the database can compile/cache the execution plan once and reuse it across multiple executions with different parameter values.
- **Cleaner code** — no manual string escaping/concatenation.

SQL Injection example (the danger of plain Statement)

```
// DANGEROUS — do not do this
String name = userInput; // e.g., userInput = "x' OR '1'='1"
String sql = "SELECT * FROM users WHERE name = '" + name + "'";
// Resulting query: SELECT * FROM users WHERE name = 'x' OR '1'='1'
// This returns ALL rows, bypassing the intended filter — a classic SQL injection.
```

Safe version with PreparedStatement

```
String sql = "SELECT * FROM users WHERE name = ?";
PreparedStatement ps = con.prepareStatement(sql);
ps.setString(1, userInput); // treated strictly as a data value, never as SQL
```

Statement vs PreparedStatement

Aspect	Statement	PreparedStatement
SQL compilation	Every execution	Compiled once, reused
Parameter binding	Manual string concatenation	`setX()` methods
SQL injection risk	High	Effectively eliminated
Performance (repeated queries)	Slower	Faster (plan caching)
Readability	Messy for dynamic queries	Clean, parameterized

Transactions [KEY]

What is a transaction?

A group of one or more database operations that must be executed **as a single unit** — either all succeed, or none take effect.

Why transactions are needed

Real-world operations often span multiple related database changes; if the process crashes partway through, the data must not be left in an inconsistent, half-updated state.

ACID (in simple language) [KEY]

Property	Meaning
Atomicity	All operations in the transaction succeed together, or all are rolled back — no partial results
Consistency	A transaction moves the database from one valid state to another, never violating constraints/rules
Isolation	Concurrent transactions don't interfere with each other's intermediate results
Durability	Once committed, changes survive even a crash immediately afterward (persisted to disk)

---

commit(), rollback(), setAutoCommit(false)

- **`setAutoCommit(false)`** — by default, JDBC commits every single statement immediately after execution; disabling auto-commit groups subsequent statements into one manual transaction.
- **`commit()`** — permanently saves all changes made since the transaction began.
- **`rollback()`** — undoes all changes made since the transaction began (used when an error occurs mid-transaction).

Banking Example — Why atomicity matters

```
Transfer ₹1000 from Account A to Account B:
  Step 1: Deduct ₹1000 from A
  Step 2: Add ₹1000 to B

If Step 1 succeeds but Step 2 fails (e.g., app crash, network error):
  → Without a transaction: ₹1000 vanishes — deducted from A, never added to B.
  → With a transaction: rollback() reverses Step 1 too — no money lost, no money created.
```

Complete JDBC Transaction Code

```java
public void transferMoney(int fromId, int toId, double amount) {
    String debitSql = "UPDATE accounts SET balance = balance - ? WHERE id = ?";
    String creditSql = "UPDATE accounts SET balance = balance + ? WHERE id = ?";

    try (Connection con = DriverManager.getConnection(url, user, password)) {
        con.setAutoCommit(false); // start manual transaction
```

```
        try (PreparedStatement debit = con.prepareStatement(debitSql);
             PreparedStatement credit = con.prepareStatement(creditSql)) {

            debit.setDouble(1, amount);
            debit.setInt(2, fromId);
            debit.executeUpdate();

            credit.setDouble(1, amount);
            credit.setInt(2, toId);
            credit.executeUpdate();

            con.commit(); // both succeeded — save permanently
            System.out.println("Transfer successful");

        } catch (SQLException e) {
            con.rollback(); // something failed — undo everything
            System.out.println("Transfer failed, rolled back");
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

Code explanation: - `con.setAutoCommit(false)` — switches from JDBC's default (commit after every statement) to explicit transaction control. - Both `debit` and `credit` updates execute inside the same `Connection` — they belong to the same transaction. - `con.commit()` runs only if both statements succeeded — makes changes permanent. - `con.rollback()` in the `catch` block undoes both operations if *either* failed, preserving atomicity.

Day 9 — Common Mistakes

- Forgetting to call `setAutoCommit(false)` and assuming statements are automatically grouped into a transaction
- Not rolling back on failure, leaving the database in an inconsistent state
- Using string concatenation for SQL instead of `PreparedStatement` (SQL injection)
- Committing too early (before all related operations complete) or too late (holding transactions open unnecessarily long, blocking other connections)

Day 9 — Interview Questions

Basic 1. **What is SQL Injection?** — An attack where untrusted input is interpreted as executable SQL, letting an attacker manipulate or extract data. 2. **How does `PreparedStatement` prevent SQL Injection?** — Parameters are bound as pure data values, never parsed as part of the SQL syntax.

Intermediate 3. **Explain ACID in one sentence each.** — See ACID table above. 4. **What does `setAutoCommit(false)` do?** — Disables JDBC's default per-statement auto-commit, letting you group multiple statements into one manual transaction.

Scenario-based 5. **In a money transfer, the debit succeeds but the credit throws an exception. What should happen?** — The transaction should `rollback()`, undoing the debit too, so no money is lost or created — this is atomicity in action.

Tricky 6. **If you never call `commit()` or `rollback()` after `setAutoCommit(false)`, what happens to your changes?** — They remain pending in the open transaction — not visible to other connections, and eventually risk locking resources; the connection must explicitly resolve the transaction.

Day 9 — Quick Revision

- `PreparedStatement` prevents SQL injection and improves performance via plan caching
- ACID = Atomicity, Consistency, Isolation, Durability
- `setAutoCommit(false)` → `commit()` on success, `rollback()` on failure
- Transactions guarantee “all or nothing” for a group of related operations
- Classic example: bank transfer — debit + credit must succeed or fail together

DAY 10 — Hibernate & ORM

ORM (Object-Relational Mapping) [KEY]

What is ORM?

A technique/library that automatically maps Java objects (classes, fields) to relational database tables (tables, columns) — and vice versa — so you work with objects in code instead of writing SQL by hand.

Why ORM?

Java is object-oriented (objects, inheritance, references); relational databases are tabular (rows, columns, foreign keys). This mismatch is called the **object-relational impedance mismatch** — ORM bridges it automatically.

Object-relational mismatch

- Objects have inheritance; tables don't naturally.
- Objects reference each other directly (`order.getCustomer()`); tables use foreign keys and joins.
- Objects can have collections (`List<Order>`); tables need a separate related table + join.

Advantages of ORM

- Eliminates repetitive JDBC boilerplate (manual `ResultSet` mapping)
- Database-vendor independence (switch MySQL → PostgreSQL with minimal code change)
- Automatic handling of relationships, caching, lazy loading
- Built-in transaction management integration

Disadvantages of ORM

- Learning curve — understanding the mapping/caching/lazy-loading behavior takes time
- Can generate inefficient SQL if used carelessly (e.g., the N+1 query problem)
- Less direct control over exact SQL compared to hand-written queries
- Debugging generated SQL can be harder than debugging code you wrote yourself

---

Hibernate

What is Hibernate?

A popular, mature Java ORM framework — the most widely used implementation of the JPA specification (see below).

Hibernate Architecture (core objects)

Component	Role
`SessionFactory`	Heavyweight, thread-safe factory — created once per application, produces `Session` objects

Component	Role
`Session`	Lightweight, single-threaded unit of work — represents a conversation between the app and the database (similar to a JDBC `Connection`, but object-oriented)
`Transaction`	Represents a unit of work to be committed or rolled back
`Query` (HQL)	Hibernate's object-oriented query language, similar to SQL but operates on entity objects/fields instead of tables/columns
`Entity`	A Java class mapped to a database table

JPA vs Hibernate [KEY] (Very important interview topic)

```
JPA (Jakarta Persistence API) = SPECIFICATION (a set of interfaces/rules — no implementati
Hibernate                     = IMPLEMENTATION (one of several libraries that fulfills the
```

Think of JPA like the `List` interface, and Hibernate like `ArrayList` — JPA defines *what* an ORM must be able to do (`persist()`, `find()`, annotations like `@Entity`), while Hibernate defines *how* it actually does it internally. Other JPA implementations exist too (EclipseLink, OpenJPA) — you can, in theory, swap Hibernate for another JPA provider without changing your `@Entity` classes.

Aspect	JPA	Hibernate
What it is	Specification/standard (interfaces + annotations)	Concrete implementation of that specification
Provides code?	No — just contracts	Yes — actual working ORM engine
Vendor-specific features	None	Yes (e.g., HQL, second-level cache specifics, `@NaturalId`)
Can you use JPA alone?	No — needs an implementation like Hibernate	Can be used with or without the JPA layer

---

JDBC vs Hibernate [KEY]

Feature	JDBC	Hibernate
SQL	Written manually	Generated automatically (or HQL/JPQL)
Mapping	Manual (`ResultSet` → object, field by field)	Automatic via annotations (`@Entity`, `@Column`)
Boilerplate	High	Low
Relationships	Manual joins and mapping	Declarative (`@OneToMany`, `@ManyToOne`, etc.)

Feature	JDBC	Hibernate
Caching	None built-in	Built-in first-level (and optional second-level) cache
Transactions	Manual (`commit()`/`rollback()`)	Managed via `Transaction`/Spring's `@Transactional`
Learning curve	Lower, but more manual work	Higher, but far less repetitive code

---

Hibernate CRUD

Entity

```java
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;

    // constructors, getters, setters
}
```

Annotation explanation: - `@Entity` — marks this class as mapped to a database table (by default, table name = class name). - `@Id` — marks the primary key field. - `@GeneratedValue` — tells the database/Hibernate to auto-generate the ID value; `GenerationType.IDENTITY` delegates ID generation to the database's auto-increment column.

Insert

```java
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();

Student student = new Student();
student.setName("Rachit");
student.setEmail("rachit@example.com");

session.persist(student); // schedules INSERT

tx.commit();
session.close();
```

Select

```java
Session session = sessionFactory.openSession();
Student student = session.find(Student.class, 1L); // primary key lookup
```

```
System.out.println(student.getName());
session.close();
```

Update

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();

Student student = session.find(Student.class, 1L);
student.setEmail("new@example.com"); // Hibernate auto-detects the change

tx.commit(); // UPDATE is issued automatically for the "dirty" field
session.close();
```

Delete

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();

Student student = session.find(Student.class, 1L);
session.remove(student);

tx.commit();
session.close();
```

Key idea (dirty checking): Once an entity is loaded within an active `Session`, Hibernate tracks changes to its fields automatically. You don't call an explicit "update" method — simply modifying the object's fields and committing the transaction is enough for Hibernate to detect the difference and issue the appropriate UPDATE SQL.

Day 10 — Common Mistakes

- Confusing JPA (the spec) with Hibernate (an implementation) — a very common interview trap
- Forgetting `@GeneratedValue`, causing manual ID assignment conflicts
- Not understanding lazy loading, leading to `LazyInitializationException` when accessing relationships outside an open session
- Assuming Hibernate always writes optimal SQL — the N+1 select problem is a frequent real-world performance bug

Day 10 — Interview Questions

Basic 1. **What is ORM?** — A technique for automatically mapping Java objects to relational database tables. 2. **What is Hibernate?** — A widely-used Java ORM framework and JPA implementation.

Intermediate 3. **Explain JPA vs Hibernate.** — JPA is a specification (interfaces/annotations); Hibernate is a concrete implementation of that specification. 4. **What is dirty checking?** — Hibernate's mechanism of tracking changes to fields of a managed entity within a session and

automatically generating the corresponding `UPDATE` SQL at commit time, without an explicit save call.

Scenario-based 5. **You load an entity, close the session, then try to access a lazily-loaded relationship. What happens?** — A `LazyInitializationException` is thrown because the session (needed to fetch the related data on demand) is no longer open.

Tricky 6. **Can you use JPA without Hibernate?** — Yes — JPA is just the specification; you could use another implementation like EclipseLink instead, though Hibernate is the most common choice.

Day 10 — Quick Revision

- ORM bridges the object-relational impedance mismatch
- `SessionFactory` (heavy, one per app) → `Session` (light, per unit of work)
- JPA = specification; Hibernate = implementation of that specification
- `@Entity` + `@Id` + `@GeneratedValue` are the minimum annotations for a mapped class
- Dirty checking auto-generates UPDATE statements for changed managed entities
- Hibernate reduces boilerplate vs raw JDBC but adds a learning curve around caching/lazy loading

DAY 11 — Spring Boot + JPA

The Layering of Persistence Technologies [KEY]

```
JDBC                    ← lowest level: raw SQL, manual mapping
 ↓
Hibernate                ← ORM engine implementing JPA, adds mapping/caching/dirty-checkin
 ↓
JPA                      ← the specification Hibernate implements (annotations, EntityMana
 ↓
Spring Data JPA           ← Spring's abstraction on top of JPA — auto-generates repository
 ↓
Spring Boot                ← auto-configures all of the above with almost zero manual setu
```

Each layer adds convenience on top of the one below it, without replacing what's underneath — Spring Data JPA still ultimately uses Hibernate, which still ultimately uses JDBC to talk to MySQL.

EntityManager [KEY]

The JPA-standard interface for interacting with the **persistence context** — the JPA equivalent of Hibernate's `Session`.

Persistence Context

An in-memory cache of managed entities within the current unit of work — Hibernate/JPA tracks these entities and their changes automatically (dirty checking).

Core operations

```java
entityManager.persist(entity);   // make a new object managed → INSERT
entityManager.find(User.class, id); // fetch by primary key → managed
entityManager.merge(detachedEntity); // reattach a detached object's changes
entityManager.remove(entity);    // schedule deletion
```

Entity Lifecycle States [KEY]

```
Transient   → a plain new object, not associated with any persistence context (just `new U
   ↓ persist()
Managed     → tracked by the persistence context; changes are auto-detected (dirty checkin
   ↓ (session/transaction ends)
Detached    → was managed, but the persistence context closed; changes are no longer track
   ↓ remove()
Removed     → scheduled for deletion; removed from the database on commit
```

State	Description
Transient	Just created with new; unknown to JPA
Managed	Attached to an active persistence context; automatically synced to the DB

State	Description
Detached	Was managed, but the context has closed; must `merge()` to resume tracking
Removed	Marked for deletion; deleted from the DB when the transaction commits

---

Spring Data JPA [KEY]

A Spring module that eliminates the need to write repository implementation code at all — you declare an interface, and Spring generates the implementation at runtime.

Repository interfaces

Interface	Provides
`CrudRepository<T, ID>`	Basic CRUD: `save()`, `findById()`, `findAll()`, `deleteById()`, etc.
`PagingAndSortingRepository<T, ID>`	Adds pagination and sorting on top of `CrudRepository`
`JpaRepository<T, ID>`	Extends `PagingAndSortingRepository` — adds JPA-specific batch operations (`flush()`, `saveAndFlush()`, etc.) — the one used in almost all real projects

```java
public interface UserRepository extends JpaRepository<User, Long> {
    // no implementation needed — Spring generates one at runtime!
    Optional<User> findByEmail(String email); // derived query, from method name
}
```

How CRUD works without manually writing SQL: At startup, Spring Data JPA generates a dynamic proxy implementation of your repository interface. Standard methods (`save`, `findById`, `delete`) map to pre-built JPA/Hibernate operations. Custom methods like `findByEmail` are parsed from the **method name itself** ("derived query methods") and translated into the equivalent JPQL/SQL automatically.

Complete Spring Boot JPA CRUD

```
Entity → Repository → Service → Controller → DTO → Exception Handler
```

Entity

```java
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;

    private String name;
    private String email;
    // getters, setters, constructors
}
```

Repository

```
public interface UserRepository extends JpaRepository<User, Long> {
}
```

Service

```
@Service
public class UserService {
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User create(User user) {
        return userRepository.save(user);
    }

    public List<User> getAll() {
        return userRepository.findAll();
    }

    public User getById(Long id) {
        return userRepository.findById(id)
                .orElseThrow(() -> new UserNotFoundException("User not found with id: " +
    }

    public User update(Long id, User updatedUser) {
        User existing = getById(id);
        existing.setName(updatedUser.getName());
        existing.setEmail(updatedUser.getEmail());
        return userRepository.save(existing);
    }

    public void delete(Long id) {
        User existing = getById(id);
        userRepository.delete(existing);
    }
}
```

Controller

```java
@RestController
@RequestMapping("/api/users")
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @PostMapping
    public ResponseEntity<User> create(@RequestBody User user) {
        return ResponseEntity.status(HttpStatus.CREATED).body(userService.create(user));
    }

    @GetMapping
    public List<User> getAll() {
        return userService.getAll();
    }

    @GetMapping("/{id}")
    public User getById(@PathVariable Long id) {
        return userService.getById(id);
    }

    @PutMapping("/{id}")
    public User update(@PathVariable Long id, @RequestBody User user) {
        return userService.update(id, user);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        userService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

The Complete Flow [KEY] (extremely common interview question)

```
Client
 ↓ HTTP Request
Controller          → maps HTTP verb/URL to a method, delegates to Service
 ↓
Service             → business logic, transaction boundary
 ↓
Repository          → Spring Data JPA interface method call
 ↓
Spring Data JPA      → generates the proxy implementation at runtime
 ↓
Hibernate            → translates the call into managed-entity operations, tracks persiste
 ↓
JDBC                 → Hibernate ultimately issues SQL through JDBC under the hood
```

```
 ↓
MySQL                → SQL executes against the actual database
 ↓ (response flows back up through every layer)
Client receives JSON response
```

Day 11 — Common Mistakes

- Injecting `UserRepository` directly into the Controller, skipping the Service layer
- Not handling the `Optional` returned by `findById()` properly (calling `.get()` blindly instead of `orElseThrow()`)
- Confusing `save()` (works for both insert and update, based on whether the ID is set) with a dedicated "insert-only" or "update-only" method
- Forgetting that `JpaRepository` methods are proxied — there's no manual implementation class to look at when debugging behavior

Day 11 — Interview Questions

Basic 1. **What is `EntityManager`?** — The JPA-standard interface for managing entities and interacting with the persistence context. 2. **What is `JpaRepository`?** — A Spring Data interface providing full CRUD plus pagination/sorting plus JPA-specific batch methods, requiring no manual implementation.

Intermediate 3. **Explain the entity lifecycle states.** — Transient → Managed → Detached → Removed (see table above). 4. **`CrudRepository` vs `JpaRepository`?** — `JpaRepository` extends `PagingAndSortingRepository` (which extends `CrudRepository`), adding pagination, sorting, and JPA-specific batch operations like `saveAndFlush()`.

Scenario-based 5. **How does Spring Data JPA know what SQL to generate for `findByEmail(String email)` without you writing any implementation?** — It parses the method name using its query-derivation convention (`findBy` + field name) and builds the equivalent JPQL query automatically at startup.

Tricky 6. **What happens if you call `save()` on an entity that already has an existing ID in the database?** — Hibernate treats it as a `merge`-like operation — updating the existing row rather than inserting a new one (assuming the entity is recognized as already persisted).

Day 11 — Quick Revision

- Layering: JDBC → Hibernate → JPA → Spring Data JPA → Spring Boot
- `EntityManager` manages the persistence context; entity states: Transient → Managed → Detached → Removed
- `JpaRepository` > `PagingAndSortingRepository` > `CrudRepository` in feature richness
- Derived query methods: Spring Data JPA parses method names into queries automatically
- Full flow: Controller → Service → Repository → Spring Data JPA → Hibernate → JDBC → MySQL

DAY 12 — JPA Relationships

Why Relationships?

Real data is connected: a `User` has one `Profile`; a `User` places many `Orders`; a `Student` enrolls in many `Courses`, and each `Course` has many `Students`. JPA lets you model these connections declaratively with annotations instead of writing manual joins.

```
User    → Profile   (one-to-one)
User    → Orders    (one-to-many)
Student → Courses   (many-to-many)
```

@OneToOne [KEY]

```
User
  |
  | 1:1
  |
Profile
```

```java
@Entity
public class User {
    @Id @GeneratedValue
    private Long id;
    private String name;

    @OneToOne(mappedBy = "user", cascade = CascadeType.ALL)
    private Profile profile;
}

@Entity
public class Profile {
    @Id @GeneratedValue
    private Long id;
    private String bio;

    @OneToOne
    @JoinColumn(name = "user_id") // foreign key column lives in this table
    private User user;
}
```

- **`@JoinColumn`** — declares the actual foreign-key column; goes on the **owning side** (the entity whose table holds the FK).
- **`mappedBy`** — placed on the **inverse side**; tells JPA "the foreign key is managed by the other entity's field," preventing a duplicate/incorrect join table from being created.
- **Cascade** — propagates operations (like persist/delete) from the parent entity to the related one (details below).

@OneToMany / @ManyToOne [KEY] (always used as a pair)

```
Department
   |
   | 1:N
   |
Employees
```

```java
@Entity
public class Department {
    @Id @GeneratedValue
    private Long id;
    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
    private List<Employee> employees = new ArrayList<>();
}

@Entity
public class Employee {
    @Id @GeneratedValue
    private Long id;
    private String name;

    @ManyToOne
    @JoinColumn(name = "department_id") // FK column lives here — Employee is the OWNING s
    private Department department;
}
```

The owning side: @ManyToOne is (almost) always the owning side, because the foreign key naturally lives on the "many" side's table (each Employee row has one department_id column). Department uses mappedBy to say "I don't own the FK; look at the department field on Employee for the relationship."

@ManyToMany [KEY]

```
Student ←→ Course
```

```java
@Entity
public class Student {
    @Id @GeneratedValue
    private Long id;
    private String name;

    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private List<Course> courses = new ArrayList<>();
}
```

```java
@Entity
public class Course {
    @Id @GeneratedValue
    private Long id;
    private String title;

    @ManyToMany(mappedBy = "courses")
    private List<Student> students = new ArrayList<>();
}
```

- **Join table:** Since neither side can hold a single foreign key (a student has many courses, and a course has many students), JPA creates a **separate join table** (`student_course`) with two foreign key columns.
- **`@JoinTable`** — declared on the **owning side** (`Student` here); specifies the join table's name and its two foreign key columns.
- **`mappedBy`** — declared on the **inverse side** (`Course`); points back to the field name (`courses`) on the owning entity.
- **Owning vs inverse side:** The owning side is whichever entity's annotation actually defines the join table structure (`@JoinTable`); the inverse side just mirrors the relationship for convenience (`mappedBy`).

Cascade [KEY]

Cascading means operations performed on a parent entity automatically propagate to its related (child) entities.

CascadeType	Effect
`PERSIST`	Saving the parent also saves any new, unsaved children
`MERGE`	Merging (updating) the parent also merges changes to children
`REMOVE`	Deleting the parent also deletes its children
`ALL`	Applies persist, merge, remove (and refresh, detach) together

Real-world example: Saving a new `Department` with a list of new `Employee` objects attached — with `CascadeType.PERSIST` (or `ALL`), calling `departmentRepository.save(department)` automatically saves the employees too, without needing to save each one manually.

Dangers of CascadeType.ALL [KEY]

Using `CascadeType.ALL` carelessly on a `@ManyToOne`/`@OneToMany` can cause **unintended mass deletion** — e.g., if `CascadeType.ALL` (which includes `REMOVE`) is placed on a relationship where the "child" entities are actually shared or independently meaningful (like `Employee` records that should be reassigned, not deleted, when a `Department` is removed), deleting the parent silently deletes far more data than intended. Best practice: apply cascade deliberately per use case, and be especially cautious with `REMOVE`/`ALL` on relationships where children have independent lifecycle significance.

Day 12 — Common Mistakes

- Putting `@JoinColumn` on both sides of a relationship (creates two conflicting foreign keys / an incorrect extra join table)
- Forgetting `mappedBy` on the inverse side, causing JPA to create a redundant join table
- Using `CascadeType.ALL` by default everywhere without considering deletion side-effects
- Creating bidirectional relationships without properly synchronizing both sides in code (e.g., adding an `Employee` to a `Department`'s list without also setting `employee.setDepartment(department)`), leading to inconsistent in-memory state before the transaction commits

Day 12 — Interview Questions

Basic 1. **What does `mappedBy` do?** — Marks the inverse (non-owning) side of a bidirectional relationship, pointing to the field on the owning entity that manages the actual foreign key/join table. 2. **What is the owning side of a relationship?** — The entity whose annotation (`@JoinColumn` or `@JoinTable`) actually defines the foreign key / join table in the database.

Intermediate 3. **Why does `@ManyToMany` need a join table?** — Because neither entity can store a single foreign key representing a many-to-many link; a separate table with two FK columns is required. 4. **Explain the difference between `@OneToMany` and `@ManyToOne`.** — They describe the same relationship from opposite sides; `@ManyToOne` is placed on the entity holding the actual foreign key column (the owning side), `@OneToMany` on the other, usually with `mappedBy`.

Scenario-based 5. **You delete a `Department` entity and all its `Employee` records get deleted too, which wasn't intended. What's the likely cause?** — `CascadeType.REMOVE` (or `ALL`) was applied on the `@OneToMany` relationship, propagating the delete to all associated employees.

Tricky 6. **Can you have `CascadeType.ALL` on a `@ManyToOne` side safely?** — Rarely appropriate — cascading a remove from the "many" side up to the "one" side (e.g., deleting an `Employee` cascading to delete the entire `Department`) is almost always unintended and dangerous.

Day 12 — Quick Revision

- `@OneToOne` → `@JoinColumn` (owning) + `mappedBy` (inverse)
- `@ManyToOne` = owning side (holds the FK); `@OneToMany` = inverse side (`mappedBy`)
- `@ManyToMany` → needs a `@JoinTable` on the owning side; `mappedBy` on the inverse side
- Cascade types: PERSIST, MERGE, REMOVE, ALL — apply carefully, especially REMOVE/ALL
- Always keep both sides of a bidirectional relationship in sync in application code

Master Comparison Tables [KEY]

These 30 comparisons cover nearly every "difference between X and Y" question asked in Java backend interviews.

1. Spring vs Spring Boot

Spring	Spring Boot
Manual configuration (XML/Java config)	Auto-configuration based on classpath
External server deployment (WAR)	Embedded server (JAR, `java -jar`)
Manual dependency version management	Curated starter dependencies
More setup time	Rapid setup

2. Spring Framework vs Spring MVC

Spring Framework	Spring MVC
The overall framework (IoC, DI, AOP, and all modules)	One specific module of Spring, for building web apps (Model-View-Controller)
Broad scope	Narrow, web-layer scope

3. Spring vs Spring Boot vs Spring MVC

Spring	Spring Boot	Spring MVC
Core framework (IoC/DI/AOP)	Auto-config + embedded server layer on top of Spring	Web module for handling HTTP requests
Foundation	Convenience layer	Specific web functionality

4. IoC vs DI

IoC	DI
A design *principle*: control of object creation is inverted	The *mechanism* used to implement IoC in Spring
Concept	Concrete technique (constructor/setter/field injection)

5. BeanFactory vs ApplicationContext

BeanFactory	ApplicationContext
Basic container, lazy bean loading	Advanced container, eager loading (by default)
Minimal features	Adds events, AOP, i18n, more
Rarely used directly today	Standard choice for all Spring apps

6. @Component vs @Service vs @Repository

@Component	@Service	@Repository
Generic stereotype	Semantic marker for business logic	Semantic marker for data access + exception translation
No extra behavior	No extra behavior (semantic only)	Adds `DataAccessException` translation

7. @Component vs @Bean

@Component	@Bean
Class-level, found via component scan	Method-level, inside `@Configuration`
For your own classes	For third-party/external classes

8. @Autowired vs Constructor Injection

@Autowired (field/setter)	Constructor Injection
Can be optional/implicit	Explicit, required by default
Harder to test standalone	Easy to test with plain `new`
Mutable fields possible	Supports `final` immutable fields

9. @Primary vs @Qualifier

@Primary	@Qualifier
Sets a default among multiple beans	Explicitly names the bean at injection point
Class-level	Injection-point level
Loses to `@Qualifier` if both present	Always wins when specified

10. Singleton vs Prototype

Singleton	Prototype
One instance per container	New instance every request
Default scope	Must be explicitly declared
Shared state risk	No shared state between usages

11. GET vs POST

GET	POST
Retrieve data	Create data
No body (typically)	Has a body
Idempotent, safe, cacheable	Not idempotent

12. PUT vs PATCH

PUT	PATCH
Full resource replacement Idempotent	Partial update Not guaranteed idempotent

13. @PathVariable vs @RequestParam

@PathVariable	@RequestParam
Part of the URL path Identifies a resource	Query string Filters/configures a request

14. @Controller vs @RestController

@Controller	@RestController
Returns view names (for server-rendered pages) Needs `@ResponseBody` per method for JSON	Returns data directly (JSON/XML) as response body `@ResponseBody` implied on all methods

15. @RequestMapping vs @GetMapping

@RequestMapping	@GetMapping
Generic, needs `method =` attribute More verbose	Shorthand specifically for GET Cleaner, more readable

16. Statement vs PreparedStatement

Statement	PreparedStatement
Compiled every execution Vulnerable to SQL injection String concatenation for parameters	Compiled once, reused Protected against SQL injection `setX()` parameter binding

17. JDBC vs Hibernate

JDBC	Hibernate
Manual SQL and mapping High boilerplate No built-in caching	Automatic SQL generation and mapping Low boilerplate Built-in first/second-level cache

18. JPA vs Hibernate

JPA	Hibernate
Specification (interfaces, annotations) No executable code	Implementation of the specification Actual working ORM engine

19. Hibernate vs Spring Data JPA

Hibernate	Spring Data JPA
The ORM engine itself	An abstraction layer that generates repository implementations on top of JPA (usually backed by Hibernate)
You write `Session`/`EntityManager` code	You just declare repository interfaces

20. CrudRepository vs JpaRepository

CrudRepository	JpaRepository
Basic CRUD operations	Extends `PagingAndSortingRepository`; adds batch ops like `saveAndFlush()`
Minimal	Feature-rich, most commonly used

21. save() vs saveAndFlush()

save()	saveAndFlush()
Persists/updates the entity; SQL may be deferred until transaction commit or next flush	Persists/updates AND immediately flushes (executes SQL right away), synchronizing with the database instantly

22. persist() vs merge()

persist()	merge()
For new (transient) entities → INSERT	For detached entities → reattaches and updates
Entity becomes managed	Returns a new managed copy; original passed-in object stays detached

23. @OneToOne vs @OneToMany

@OneToOne	@OneToMany
One record relates to exactly one other record	One record relates to many records
Example: User ↔ Profile	Example: Department → Employees

24. @OneToMany vs @ManyToOne

@OneToMany	@ManyToOne
Inverse side (usually `mappedBy`)	Owning side (usually holds `@JoinColumn`/FK)
“One” entity’s perspective	“Many” entity’s perspective

25. @ManyToMany vs @OneToMany

@ManyToMany	@OneToMany
Needs a join table (`@JoinTable`)	Uses a direct foreign key on the “many” side
Both sides can have multiple related records	Only the “many” side can have multiple

26. Cascade vs OrphanRemoval

Cascade	orphanRemoval
Propagates operations (persist/merge/remove) from parent to child	Specifically deletes a child when it’s *removed from the parent’s collection*, even without deleting the parent itself
Triggered by explicit operations on the parent	Triggered by the child losing its association

27. EntityManager vs Session

EntityManager	Session
JPA-standard interface	Hibernate-specific interface (Hibernate’s native API)
Portable across JPA providers	Tied to Hibernate
`persist()`, `find()`, `merge()`, `remove()`	`persist()`, `find` (via get/load historically), similar operations

28. Checked vs Unchecked Exception

Checked Exception	Unchecked Exception
Must be declared (`throws`) or caught at compile time	No compile-time enforcement
Extends `Exception` (not `RuntimeException`)	Extends `RuntimeException`
Example: `SQLException`, `IOException`	Example: `NullPointerException`, custom `RuntimeExceptions`

29. @ExceptionHandler vs @RestControllerAdvice

@ExceptionHandler	@RestControllerAdvice
Local to one controller	Global, across all controllers
Duplicated per class if reused	Centralized, written once

30. System.out.println vs Logger

System.out.println	Logger (SLF4J)
Always prints, not configurable	Configurable levels, destinations
No context (class, level, timestamp)	Structured, leveled, production-grade

Complete Project — Employee Management REST API

This project combines every concept from all 12 days into one working Spring Boot backend.

Architecture

```
Client
   ↓
Controller     → HTTP mapping, delegates to Service
   ↓
DTO            → shapes the request/response contract
   ↓
Service        → business logic, transactions
   ↓
Repository     → Spring Data JPA interface
   ↓
JPA / Hibernate → ORM mapping, dirty checking, persistence context
   ↓
JDBC            → underlying SQL execution
   ↓
MySQL
```

Project Structure

```
src/main/java/com/example/employee/
├── controller/
│   └── EmployeeController.java
├── service/
│   └── EmployeeService.java
├── repository/
│   └── EmployeeRepository.java
├── entity/
│   ├── Employee.java
│   └── Department.java
├── dto/
│   ├── EmployeeRequestDto.java
│   └── EmployeeResponseDto.java
├── exception/
│   ├── EmployeeNotFoundException.java
│   └── GlobalExceptionHandler.java
└── EmployeeApplication.java
```

application.properties

```
server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/employeedb
spring.datasource.username=root
spring.datasource.password=password

spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true

logging.level.org.springframework=INFO
```

Entities

```java
@Entity
@Table(name = "departments")
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.PERSIST)
    private List<Employee> employees = new ArrayList<>();

    // getters/setters
}

@Entity
@Table(name = "employees")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private Double salary;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;

    // getters/setters
}
```

DTOs

```java
public class EmployeeRequestDto {
    @NotBlank(message = "Name is required")
    private String name;

    @Email(message = "Email must be valid")
    private String email;

    @Positive(message = "Salary must be positive")
    private Double salary;
```

```
    private Long departmentId;
    // getters/setters
}

public class EmployeeResponseDto {
    private Long id;
    private String name;
    private String email;
    private Double salary;
    private String departmentName;
    // constructor, getters/setters
}
```

Repository

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    Page<Employee> findByNameContainingIgnoreCase(String name, Pageable pageable);
}
```

Custom Exception

```
public class EmployeeNotFoundException extends RuntimeException {
    public EmployeeNotFoundException(String message) {
        super(message);
    }
}
```

Service

```
@Service
public class EmployeeService {
    private static final Logger log = LoggerFactory.getLogger(EmployeeService.class);
    private final EmployeeRepository employeeRepository;

    public EmployeeService(EmployeeRepository employeeRepository) {
        this.employeeRepository = employeeRepository;
    }

    public EmployeeResponseDto create(EmployeeRequestDto dto) {
        Employee employee = new Employee();
        employee.setName(dto.getName());
        employee.setEmail(dto.getEmail());
        employee.setSalary(dto.getSalary());

        Employee saved = employeeRepository.save(employee);
        log.info("Created employee with id={}", saved.getId());
        return toResponseDto(saved);
    }

    public Page<EmployeeResponseDto> getAll(int page, int size) {
```

```
        return employeeRepository.findAll(PageRequest.of(page, size))
                .map(this::toResponseDto);
    }

    public EmployeeResponseDto getById(Long id) {
        Employee employee = employeeRepository.findById(id)
                .orElseThrow(() -> new EmployeeNotFoundException("Employee not found with
        return toResponseDto(employee);
    }

    public EmployeeResponseDto update(Long id, EmployeeRequestDto dto) {
        Employee employee = employeeRepository.findById(id)
                .orElseThrow(() -> new EmployeeNotFoundException("Employee not found with
        employee.setName(dto.getName());
        employee.setEmail(dto.getEmail());
        employee.setSalary(dto.getSalary());
        return toResponseDto(employeeRepository.save(employee));
    }

    public void delete(Long id) {
        if (!employeeRepository.existsById(id)) {
            throw new EmployeeNotFoundException("Employee not found with id: " + id);
        }
        employeeRepository.deleteById(id);
        log.info("Deleted employee with id={}", id);
    }

    public Page<EmployeeResponseDto> search(String name, int page, int size) {
        return employeeRepository
                .findByNameContainingIgnoreCase(name, PageRequest.of(page, size))
                .map(this::toResponseDto);
    }

    private EmployeeResponseDto toResponseDto(Employee employee) {
        EmployeeResponseDto dto = new EmployeeResponseDto();
        dto.setId(employee.getId());
        dto.setName(employee.getName());
        dto.setEmail(employee.getEmail());
        dto.setSalary(employee.getSalary());
        if (employee.getDepartment() != null) {
            dto.setDepartmentName(employee.getDepartment().getName());
        }
        return dto;
    }
}
```

Controller

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {
    private final EmployeeService employeeService;
```

```
    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    @PostMapping
    public ResponseEntity<EmployeeResponseDto> create(@Valid @RequestBody EmployeeRequestD
        return ResponseEntity.status(HttpStatus.CREATED).body(employeeService.create(dto))
    }

    @GetMapping
    public Page<EmployeeResponseDto> getAll(
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "10") int size) {
        return employeeService.getAll(page, size);
    }

    @GetMapping("/{id}")
    public EmployeeResponseDto getById(@PathVariable Long id) {
        return employeeService.getById(id);
    }

    @PutMapping("/{id}")
    public EmployeeResponseDto update(@PathVariable Long id, @Valid @RequestBody EmployeeR
        return employeeService.update(id, dto);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        employeeService.delete(id);
        return ResponseEntity.noContent().build();
    }

    @GetMapping("/search")
    public Page<EmployeeResponseDto> search(
            @RequestParam String name,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "10") int size) {
        return employeeService.search(name, page, size);
    }
}
```

Global Exception Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(EmployeeNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleNotFound(EmployeeNotFoundException ex
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", 404);
        body.put("message", ex.getMessage());
```

```
        body.put("path", request.getRequestURI());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidation(MethodArgumentNotValidExce
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", 400);
        body.put("errors", ex.getBindingResult().getFieldErrors().stream()
                .collect(Collectors.toMap(FieldError::getField, FieldError::getDefaultMess
        return ResponseEntity.badRequest().body(body);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleGeneric(Exception ex, HttpServletRequ
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", 500);
        body.put("message", "Something went wrong");
        body.put("path", request.getRequestURI());
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(body);
    }
}
```

Main Class

```
@SpringBootApplication
public class EmployeeApplication {
    public static void main(String[] args) {
        SpringApplication.run(EmployeeApplication.class, args);
    }
}
```

API Summary

Method	Endpoint	Purpose
POST	/api/employees	Create a new employee
GET	/api/employees?page=0&size=10	Get paginated employees
GET	/api/employees/{id}	Get employee by ID
PUT	/api/employees/{id}	Update an employee
DELETE	/api/employees/{id}	Delete an employee
GET	/api/employees/search?name=...	Search employees by name

What this project demonstrates: layered architecture, DTO usage, Bean Validation, pagination, @ManyToOne/@OneToMany relationships, global exception handling, structured error responses, and logging — everything covered across the 12 days, combined into one coherent, interview-defensible codebase.

100 Spring Boot Interview Questions

Beginner (30)

1. What is Spring Framework? Short: A lightweight Java framework built around IoC and DI for enterprise application development. Detail: Spring manages object creation and wiring so developers focus on business logic instead of plumbing code; it's modular (Core, MVC, Data, Security, AOP). Follow-up: What problem does it solve compared to plain Java/EJB?

2. What is Spring Boot? Short: A framework on top of Spring providing auto-configuration, embedded servers, and starter dependencies. Detail: It removes manual setup overhead — classpath-based auto-configuration and starters let you build a production-ready app with minimal code. Follow-up: What does `@SpringBootApplication` combine?

3. What is IoC (Inversion of Control)? Short: A principle where object creation/wiring is handled by a container instead of application code. Detail: Instead of `new SomeClass()`, the Spring container creates and manages the object's lifecycle, injecting required dependencies. Follow-up: How is IoC implemented in Spring?

4. What is Dependency Injection? Short: The mechanism by which the IoC container supplies an object's dependencies rather than the object creating them itself. Detail: Supports constructor, setter, and field injection; constructor is preferred for immutability and testability. Follow-up: Why is constructor injection preferred?

5. What is a Spring Bean? Short: An object instantiated, configured, and managed by the Spring IoC container. Detail: Beans are created via component scanning (`@Component`) or explicit @Bean methods, and their lifecycle is fully managed by the container. Follow-up: What is the default bean scope?

6. What is `@Component`? Short: Marks a class as a Spring-managed bean, detected via component scanning. Detail: The base stereotype annotation; `@Service`, `@Repository`, `@Controller` all build on it. Follow-up: What's the difference from @Bean?

7. What is `@Autowired`? Short: Triggers automatic dependency resolution and injection by type. Detail: Works on constructors, setters, or fields; optional on a single constructor since Spring 4.3. Follow-up: What happens if two matching beans exist?

8. What is `@Service`? Short: A semantic stereotype marking a class as containing business logic. Detail: Functionally equivalent to `@Component`, but communicates intent clearly to other developers. Follow-up: Is it functionally different from `@Component`?

9. What is `@Repository`? Short: Marks a data-access class; also enables Spring's exception translation. Detail: Converts low-level persistence exceptions into Spring's unified `DataAccessException` hierarchy. Follow-up: What extra behavior does it add beyond `@Component`?

10. What is `@Bean`? Short: A method-level annotation inside `@Configuration` used to manually declare a bean. Detail: Used for third-party classes you can't annotate directly with `@Component`. Follow-up: When would you prefer @Bean over `@Component`?

11. What is Spring Boot auto-configuration? Short: Automatic bean configuration based on what's present on the classpath. Detail: Uses conditional annotations (`@ConditionalOnClass`, `@ConditionalOnMissingBean`) to configure sensible defaults, backing off if you define your own bean. Follow-up: How can you exclude a specific auto-configuration?

12. How does a REST request flow through Spring Boot? Short: Client → Tomcat → DispatcherServlet → HandlerMapping → Controller → Service → Repository → DB → back up through Jackson to JSON. Detail: `DispatcherServlet` is the front controller intercepting all requests and routing them appropriately. Follow-up: What is `DispatcherServlet`?

13. What is `DispatcherServlet`? Short: Spring MVC's front controller — the single entry point for all incoming HTTP requests. Detail: It delegates to `HandlerMapping` to find the right controller method, then to the controller, then handles the response conversion. Follow-up: What design pattern does this represent?

14. What is `ResponseEntity`? Short: A wrapper giving full control over an HTTP response's status code, headers, and body. Detail: Without it, returning a plain object always implies `200 OK`, which is often semantically wrong (e.g., for creation, deletion). Follow-up: How would you return a `201 Created`?

15. Difference between PUT and PATCH? Short: PUT fully replaces a resource; PATCH partially updates it. Detail: PUT is idempotent by convention; PATCH is not guaranteed to be. Follow-up: Is PATCH always idempotent?

16. Difference between `@PathVariable` and `@RequestParam`? Short: `@PathVariable` extracts values from the URL path (identifies a resource); `@RequestParam` extracts values from the query string (filters/configures). Detail: `/users/10` uses path variable; `/users?age=25` uses request param. Follow-up: When would each be required vs optional?

17. How does exception handling work in Spring Boot? Short: Via `@ExceptionHandler` (local) or `@RestControllerAdvice` (global) methods that intercept thrown exceptions and return structured error responses. Detail: Prevents leaking raw stack traces and ensures consistent error format. Follow-up: What's the difference between the two?

18. What is `@RestControllerAdvice`? Short: Provides centralized, global exception handling across all controllers. Detail: Combines `@ControllerAdvice` + `@ResponseBody`. Follow-up: What if a controller has its own local `@ExceptionHandler` too?

19. What is JDBC? Short: A standard Java API for connecting to and interacting with relational databases. Detail: Provides interfaces (`Connection`, `Statement`, `ResultSet`) implemented by vendor-specific drivers. Follow-up: What's the JDBC architecture?

20. What is `PreparedStatement`? Short: A precompiled, parameterized SQL statement that prevents SQL injection and improves performance. Detail: Parameters bound via `setX()` are treated strictly as data, never as executable SQL. Follow-up: How exactly does it prevent SQL injection?

21. What is SQL injection? Short: An attack where untrusted input is interpreted as part of the SQL query itself, letting an attacker manipulate the query. Detail: Happens when raw user input is concatenated directly into SQL strings. Follow-up: How do you prevent it?

22. What is a database transaction? Short: A group of operations executed as a single all-or-nothing unit. Detail: Ensures partial failures don't leave the database in an inconsistent state. Follow-up: What does ACID stand for?

23. Explain ACID. Short: Atomicity, Consistency, Isolation, Durability — the four guarantees of a reliable transaction. Detail: Atomicity = all-or-nothing; Consistency = valid state transitions; Isolation = concurrent transactions don't interfere; Durability = committed data survives crashes. Follow-up: Give a real-world example needing atomicity.

24. What is Hibernate? Short: A popular Java ORM framework and the most widely-used implementation of JPA. Detail: Automatically maps Java objects to database tables, handling SQL generation, caching, and dirty checking. Follow-up: What is JPA in relation to Hibernate?

25. What is JPA? Short: Jakarta Persistence API — a specification (set of interfaces/annotations) for ORM in Java, not an implementation itself. Detail: Hibernate is one implementation of the JPA spec; others exist (EclipseLink). Follow-up: Difference between JPA and Hibernate?

26. Difference between JPA and Hibernate? Short: JPA is the specification; Hibernate is a concrete implementation of it. Detail: You code against JPA annotations/interfaces, but

Hibernate does the actual work underneath. Follow-up: Can you swap Hibernate for another JPA provider?

27. What is Spring Data JPA? Short: A Spring abstraction that auto-generates repository implementations on top of JPA. Detail: You declare an interface extending `JpaRepository`; Spring provides the implementation at runtime. Follow-up: How does it know what SQL to run for a derived method name?

28. What is `EntityManager`? Short: The JPA-standard interface for managing entities and the persistence context. Detail: Provides `persist()`, `find()`, `merge()`, `remove()` operations. Follow-up: What is the persistence context?

29. What is `JpaRepository`? Short: A Spring Data interface providing full CRUD, pagination, sorting, and JPA-specific batch operations. Detail: Extends `PagingAndSortingRepository`, which extends `CrudRepository`. Follow-up: Difference from `CrudRepository`?

30. Explain the entity lifecycle. Short: Transient → Managed → Detached → Removed. Detail: Transient = plain new object; Managed = tracked by persistence context; Detached = context closed, changes untracked; Removed = scheduled for deletion. Follow-up: What happens if you modify a detached entity?

Intermediate (40)

31. Why is constructor injection preferred over field injection? Short: Immutability, testability, fail-fast on missing dependencies. Detail: `final` fields, no reflection needed, dependencies visible in the constructor signature, and objects are never in a partially-initialized state.

32. What is `@Primary` and when do you use it? Short: Marks a default bean when multiple candidates exist for autowiring. Detail: Applied at the class/bean-definition level; loses to `@Qualifier` when both are specified at an injection point.

33. What is `@Qualifier` and how does it differ from `@Primary`? Short: Explicitly names which bean to inject at a specific injection point. Detail: `@Qualifier` always wins over `@Primary` because it's more specific.

34. Explain Bean scopes in Spring. Short: singleton (default, one per container), prototype (new instance every request), request/session (web-specific). Detail: Choosing prototype for stateful, non-thread-safe beans avoids shared-state bugs common with singleton.

35. What is the Bean lifecycle? Short: Definition → Instantiation → DI → Initialization (`@PostConstruct`) → Ready → Destruction (`@PreDestroy`). Detail: `@PostConstruct` runs after full dependency injection; `@PreDestroy` runs on graceful shutdown.

36. Difference between `@Component` and `@Bean`? Short: `@Component` is class-level, detected by scanning; `@Bean` is method-level inside `@Configuration`, used for external classes.

37. What extra behavior does `@Repository` add? Short: Automatic translation of persistence exceptions into Spring's `DataAccessException` hierarchy.

38. Explain `@SpringBootApplication`'s composition. Short: `@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan`. Detail: This single annotation bootstraps configuration, classpath-driven auto-config, and package scanning.

39. What are Spring Boot starters and why are they useful? Short: Curated dependency bundles for a feature area, with tested-compatible versions. Detail: Avoids manual version-compatibility guesswork across dozens of libraries.

40. Difference between `CommandLineRunner` and `ApplicationRunner`? Short: `CommandLineRunner` gets raw `String[]` args; `ApplicationRunner` gets parsed `ApplicationArguments`.

41. How does `@RequestBody` work internally? Short: Jackson (an `HttpMessageConverter`) deserializes the incoming JSON into the declared Java type.

42. Why use `ResponseEntity` instead of returning a plain object? Short: Gives explicit control over status code and headers rather than always defaulting to 200 OK.

43. What is a DTO and why use one? Short: A plain object for transferring data between layers, decoupled from the database entity. Detail: Prevents leaking internal fields, allows independent validation rules per operation (create vs update).

44. Explain the internal REST request flow in detail. Short: Tomcat → DispatcherServlet → HandlerMapping → Controller → Service → Repository → DB → Jackson → JSON response.

45. Difference between `@ExceptionHandler` and `@RestControllerAdvice`? Short: `@ExceptionHandler` alone is local to one controller; `@RestControllerAdvice` applies globally across all controllers.

46. Why create custom exceptions instead of using generic `RuntimeException`? Short: Clearer intent and precise mapping to specific HTTP status codes in the global handler.

47. Why use try-with-resources with JDBC? Short: Guarantees automatic closing of `Connection`/`Statement`/`ResultSet`, preventing connection leaks even on exceptions.

48. Difference between `Statement` and `PreparedStatement`? Short: `Statement` recompiles SQL every time and is vulnerable to SQL injection; `PreparedStatement` is precompiled, parameterized, and safe.

49. What does `setAutoCommit(false)` do? Short: Disables JDBC's default per-statement auto-commit, allowing multiple statements to be grouped into one manual transaction.

50. Explain `commit()` and `rollback()`. Short: `commit()` permanently saves all changes since the transaction began; `rollback()` undoes them if something failed.

51. What is dirty checking in Hibernate? Short: Hibernate automatically detects changes to fields of a managed entity and generates the corresponding UPDATE SQL at commit — no explicit save call needed.

52. JDBC vs Hibernate — key differences? Short: JDBC requires manual SQL and mapping with no caching; Hibernate auto-generates SQL, handles mapping/relationships declaratively, and provides built-in caching.

53. What is `SessionFactory` vs `Session`? Short: `SessionFactory` is a heavyweight, thread-safe factory created once per app; `Session` is a lightweight, single-threaded unit of work.

54. `CrudRepository` vs `JpaRepository`? Short: `JpaRepository` extends `PagingAndSortingRepository` (which extends `CrudRepository`), adding pagination, sorting, and batch operations like `saveAndFlush()`.

55. How does Spring Data JPA generate queries from method names? Short: It parses conventions like `findBy<FieldName>` and builds the equivalent JPQL query automatically at startup — no manual implementation needed.

56. `save()` vs `saveAndFlush()`? Short: `save()` may defer SQL execution until commit or next flush; `saveAndFlush()` executes SQL immediately.

57. `persist()` vs `merge()`? Short: `persist()` is for new (transient) entities → INSERT; `merge()` reattaches a detached entity's changes, returning a new managed copy.

58. What is `mappedBy` used for? Short: Marks the inverse side of a bidirectional relationship, pointing to the field on the owning entity that manages the actual foreign key.

59. What is the owning side of a JPA relationship? Short: The entity whose annotation (`@JoinColumn`/`@JoinTable`) actually defines the foreign key or join table in the database.

60. Explain `@JoinColumn` vs `@JoinTable`. Short: `@JoinColumn` declares a direct foreign-key column (used for `@OneToOne`/`@ManyToOne`); `@JoinTable` declares a separate join table (used for `@ManyToMany`).

61. What is Cascade in JPA? Short: Propagates operations (persist, merge, remove) performed on a parent entity to its related child entities.

62. Dangers of `CascadeType.ALL`? Short: Can cause unintended mass deletion if applied carelessly on relationships where children have independent lifecycle significance.

63. Difference between checked and unchecked exceptions? Short: Checked exceptions must be declared/caught at compile time (extend `Exception`); unchecked don't require this (extend `RuntimeException`).

64. Why use SLF4J Logger instead of `System.out.println`? Short: Configurable log levels and destinations, production-safe, avoids unnecessary string building when a level is disabled.

65. What are the standard log levels? Short: DEBUG < INFO < WARN < ERROR, in increasing severity.

66. What does `@ConfigurationProperties` do? Short: Binds a group of related `application.properties` values into a strongly-typed POJO, instead of individual `@Value` injections.

67. How do Spring Profiles work? Short: Environment-specific configuration files (`application-dev.properties`, `application-prod.properties`) activated via `spring.profiles.active`.

68. What is `@Valid` used for in a controller? Short: Triggers Bean Validation (e.g., `@NotBlank`, `@Email`) on the annotated `@RequestBody` parameter, throwing `MethodArgumentNotValidException` on failure.

69. How would you implement pagination in Spring Data JPA? Short: Accept `Pageable`/`PageRequest` parameters in the repository method, returning a Page<T>.

70. What is `@Transactional` and why is it important? Short: Declares a method as running inside a database transaction — Spring automatically commits on success or rolls back on a runtime exception.

Advanced (20)

71. Explain the N+1 select problem and how to avoid it. Short: Fetching a parent list, then lazily loading each child collection separately, causes 1 query for the parents + N queries for children. Detail: Fixed with `JOIN FETCH` in JPQL, `@EntityGraph`, or eager fetch strategies applied deliberately for the specific query.

72. How does Hibernate's first-level cache work? Short: Every `Session`/persistence context automatically caches entities it has loaded or persisted, so repeated lookups by ID within the same session don't re-hit the database. Detail: Scoped to a single `Session`; cleared when the session closes. Second-level cache (optional, e.g., Ehcache) spans across sessions.

73. What is `LazyInitializationException` and when does it occur? Short: Thrown when code tries to access a lazily-loaded association after its owning session has closed. Detail: Common when returning entities directly from a controller after the transaction/session has ended — fix with DTOs, `JOIN FETCH`, or keeping the session open appropriately (not `OpenSessionInView` as a long-term fix, since it hides the underlying design issue).

74. Explain how `@Transactional` works internally. Short: Spring wraps the annotated method (and its call chain within the same proxy) in an AOP proxy that begins a transaction before the method executes and commits/rolls back after. Detail: Only works on public methods called from outside the class (self-invocation bypasses the proxy) by default, using the standard Spring AOP proxy mechanism.

75. What is Spring AOP and how does it relate to `@Transactional`/logging/security? Short: Aspect-Oriented Programming lets Spring inject cross-cutting behavior (transactions, logging, security checks) around method calls via dynamically generated proxies, without modifying business logic code.

76. Explain the difference between JPA's `EntityManager` and Hibernate's native `Session`. Short: `EntityManager` is the portable, standard JPA interface; `Session` is Hibernate's own native API (superset of JPA, with Hibernate-specific extras) — a `Session` can actually be unwrapped from an `EntityManager` when using Hibernate as the provider.

77. How does connection pooling improve performance, and what does Spring Boot use by default? Short: Reuses a pool of pre-established database connections instead of opening/closing a new physical connection per request; Spring Boot defaults to HikariCP.

78. Explain optimistic vs pessimistic locking in JPA. Short: Optimistic locking (`@Version`) detects conflicts at commit time by checking a version column, rolling back if it changed; pessimistic locking acquires a database-level lock upfront, blocking other transactions from touching the row.

79. What is the difference between `@Async` and normal synchronous method calls in Spring? Short: `@Async` runs the method on a separate thread from a task executor, returning immediately (optionally with a `Future`/`CompletableFuture`), rather than blocking the caller's thread.

80. Explain how Spring Security's filter chain relates to `DispatcherServlet`. Short: Spring Security registers a chain of Servlet filters that run *before* `DispatcherServlet`, intercepting requests for authentication/authorization checks before they ever reach a controller.

81. What is the difference between `@RequestMapping(method = GET)` and `@GetMapping`, at the bytecode/annotation level? Short: `@GetMapping` is a meta-annotation composed of `@RequestMapping(method = RequestMethod.GET)` — functionally identical, purely a readability shorthand.

82. How would you handle circular dependencies between two Spring beans? Short: Prefer redesigning to remove the cycle (often a sign of poor separation of concerns); if unavoid-

able, Spring can resolve some cycles via setter injection or `@Lazy`, but constructor-injected cycles fail at startup by design (fail-fast).

83. Explain how Hibernate translates a `@ManyToMany` bidirectional relationship into SQL. Short: A join table with two foreign key columns is created; adding/removing from the collection on the owning side inserts/deletes rows in that join table.

84. What's the difference between `orphanRemoval = true` and `CascadeType.REMOVE`? Short: `CascadeType.REMOVE` deletes children when the *parent itself* is deleted; `orphanRemoval` deletes a child specifically when it's *removed from the parent's collection*, even if the parent still exists.

85. How does Spring Boot decide the order of multiple `CommandLineRunner` beans? Short: By the `@Order` annotation (or `Ordered` interface) on each bean; without it, order is not guaranteed.

86. What is idempotency and why does it matter for API design under retries? Short: An idempotent operation produces the same result no matter how many times it's executed; critical for safe client-side retries on network failures (e.g., retrying a PUT is safe, retrying a non-idempotent POST could create duplicates).

87. Explain how JPQL differs from native SQL queries in Spring Data JPA. Short: JPQL operates on entity objects and their fields (not raw table/column names), making queries portable across databases; native SQL is direct, database-specific SQL, used via `@Query(nativeQuery = true)` when JPQL can't express something.

88. What is the difference between `@ControllerAdvice` and `@RestControllerAdvice`? Short: `@RestControllerAdvice` = `@ControllerAdvice` + `@ResponseBody`, meaning handler methods return response bodies directly (JSON), matching REST API needs; plain `@ControllerAdvice` targets traditional MVC apps that return view names.

89. How would you design database schema migrations in a Spring Boot project (vs `ddl-auto=update`)? Short: Use a dedicated migration tool (Flyway or Liquibase) with versioned SQL scripts for production, rather than relying on Hibernate's `ddl-auto`, which is convenient for development but risky/unpredictable for production schema changes.

90. Explain how Spring resolves ambiguity when injecting a `List<SomeInterface>`. Short: Instead of failing on multiple matching beans, Spring injects *all* matching beans into the collection automatically, ordered by `@Order` if specified — useful for strategy-pattern-style designs.

Scenario-Based (10)

91. Your `/api/users` endpoint is slow under load. How would you investigate and fix it? Approach: Check for the N+1 query problem (enable `spring.jpa.show-sql`), verify connection pool sizing (HikariCP settings), check for missing database indexes on frequently filtered columns, and profile whether the bottleneck is DB, network, or application logic (e.g., unnecessary synchronous calls).

92. A teammate's PUT endpoint silently creates a new record instead of updating an existing one when the ID doesn't exist. What's wrong and how do you fix it? Approach: The service isn't checking existence before saving. Fix: fetch the entity by ID first and throw a `NotFoundException` (mapped to 404) if absent, rather than blindly calling `save()` with a client-supplied ID.

93. Your team wants zero-downtime deployment, but `ddl-auto=update` sometimes locks tables during startup on a large production database. What would you recommend? Approach: Switch to Flyway/Liquibase for controlled, versioned migrations run separately from application startup, and set `ddl-auto=validate` (or `none`) in production so Hibernate never attempts automatic schema changes.

94. Two microservices both write to the same `orders` table and you're seeing intermittent inconsistent balances. What's likely happening, and what's the fix? Approach: Likely a race condition from concurrent transactions without proper isolation/locking. Fix: use appropriate transaction isolation levels, or optimistic locking (`@Version`) to detect and retry conflicting concurrent updates, or pessimistic locking if conflicts are frequent and must be prevented outright.

95. A client complains your DELETE endpoint returns `200 OK` with a JSON body, but their tooling expects `204 No Content`. How do you fix this without breaking existing clients who rely on the body? Approach: Introduce API versioning (e.g., `/api/v2/employees/{id}`) or a content-negotiation approach, rather than silently changing behavior for existing consumers — a direct behavior change on a live endpoint is a breaking change.

96. Your global exception handler returns a generic 500 error for a validation failure that should be a 400. What's the likely cause? Approach: The `@ExceptionHandler(Exception.class)` catch-all is likely defined *before* (or is catching too broadly compared to) the more specific validation exception handler; ensure `MethodArgumentNotValidException` has its own dedicated handler mapped to 400, and that `@Valid` is actually present on the controller parameter.

97. A new developer complains that autowiring a service fails with `NoUniqueBeanDefinitionException` after they added a second implementation of an existing interface. How do you guide them? Approach: Explain the ambiguity, then show them the two fixes — `@Primary` on the default implementation, or `@Qualifier("beanName")` at each injection point — and discuss which fits their use case (a true default vs. genuinely needing to pick per-context).

98. You need to support both a `User` created via self-registration and via admin invite, with different required fields. How would you structure your DTOs and validation? Approach: Use separate DTOs (`SelfRegistrationDto`, `AdminInviteDto`) with different validation annotations per field's requirements, rather than one bloated DTO with conditionally-required fields — keeps validation rules explicit and avoids fragile conditional logic.

99. Your `@OneToMany` relationship between `Department` and `Employee` uses `CascadeType.ALL`, and deleting a department accidentally deleted employees that should have been reassigned instead. How do you redesign this? Approach: Remove `REMOVE`/`ALL` cascade from this relationship; instead, explicitly reassign or nullify

`employee.department` before deleting the department, making the intentional business rule explicit in the service layer rather than relying on cascade side-effects.

100. Interviewer asks you to design the full request flow for creating an order that also updates inventory, and explain how you'd guarantee consistency. Approach: Both operations (create order + decrement inventory) belong in the same `@Transactional` service method so they commit or roll back together; discuss isolation level considerations for concurrent inventory decrements (optimistic locking with `@Version` on the inventory row is a strong, scalable answer), and mention idempotency if the operation might be retried by a client.

Rapid Revision Cheat Sheet

Spring Core

- **IoC** — control of object creation moves from your code to a container
- **DI** — the mechanism implementing IoC (constructor preferred)
- **Bean** — an object managed by the Spring container
- **Container** — `ApplicationContext` (standard) or `BeanFactory` (basic, rarely used)
- `@Component` — generic bean; `@Service`/`@Repository`/`@Controller` — semantic stereotypes
- `@Bean` — manual bean declaration inside `@Configuration`, for third-party classes
- `@Autowired` — automatic dependency resolution by type
- `@Primary` — default bean among multiple candidates
- `@Qualifier` — explicit bean choice at injection point (wins over `@Primary`)

Spring Boot

- `@SpringBootApplication = @SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan`
- **Starters** — curated, version-compatible dependency bundles
- **Auto-configuration** — classpath-driven, conditional bean setup
- `application.properties`/`.yml` — externalized configuration
- **Embedded server** — Tomcat bundled in the JAR; `java -jar app.jar` runs it all

REST

- HTTP methods: GET (read), POST (create), PUT (full update), PATCH (partial update), DELETE (remove)
- Status codes: 2xx success, 4xx client error, 5xx server error
- `@RestController` = `@Controller` + `@ResponseBody`
- `@GetMapping`/`@PostMapping`/etc. = shorthand for `@RequestMapping(method=...)`
- `@PathVariable` — identifies a resource; `@RequestParam` — filters/configures
- `ResponseEntity<T>` — full control over status code + headers + body
- JSON conversion handled automatically by Jackson

Exception Handling & Logging

- `@ExceptionHandler` — local to one controller
- `@RestControllerAdvice` — global, centralized (preferred)
- Custom exceptions communicate intent, map to specific HTTP statuses
- Never expose raw stack traces to clients
- Logger (SLF4J) > `System.out.println`; levels: DEBUG < INFO < WARN < ERROR

JDBC

- `DriverManager` → `Connection` → `Statement`/`PreparedStatement` → `ResultSet`
- Always use `PreparedStatement` (prevents SQL injection, better performance)
- try-with-resources auto-closes JDBC resources
- `executeQuery()` → SELECT; `executeUpdate()` → INSERT/UPDATE/DELETE
- **ACID:** Atomicity, Consistency, Isolation, Durability
- `setAutoCommit(false)` → `commit()` on success, `rollback()` on failure

Hibernate/JPA

- **JPA = specification; Hibernate = implementation** (this distinction is asked constantly)
- `SessionFactory` (heavy, once per app) → `Session` (light, per unit of work)
- `EntityManager` — JPA-standard interface for managing the persistence context
- Entity lifecycle: **Transient → Managed → Detached → Removed**
- `JpaRepository` > `PagingAndSortingRepository` > `CrudRepository`
- Derived query methods — Spring Data JPA parses method names into JPQL automatically
- `persist()` — new entity → INSERT; `merge()` — detached entity → reattach/update
- `save()` — may defer SQL; `saveAndFlush()` — executes immediately
- **Relationships:** `@OneToOne` (+ `@JoinColumn`/`mappedBy`), `@ManyToOne` (owning side, holds FK), `@OneToMany` (inverse, `mappedBy`), `@ManyToMany` (+ `@JoinTable`)
- **Cascade:** PERSIST, MERGE, REMOVE, ALL — use REMOVE/ALL carefully
- Full flow: **Controller → Service → Repository → Spring Data JPA → Hibernate → JDBC → MySQL**

Final Mock Interview — 50 Questions, 10 Rounds

For each question: what a strong candidate says, common mistakes, and a model answer.

Round 1 — Basic Java/Spring

1. What's the difference between an interface and an abstract class, and why does this matter for Spring's DI? Good candidate: Explains interfaces define pure contracts (no state), abstract classes can share implementation — and ties it to Spring: depending on an interface (not a concrete/abstract class) is what enables loose coupling and swappable implementations. Common mistake: Giving a generic Java answer with no connection to DI. Model answer: "Interfaces define a pure contract with no implementation; abstract classes can hold shared state/logic. In Spring, we design dependencies around interfaces so beans can be swapped or mocked without touching dependent classes — that's the foundation of loose coupling."

2. Why does Spring favor composition (DI) over inheritance for reusability? Model answer: "Inheritance creates tight, compile-time coupling to a parent's implementation. Composition lets you assemble behavior from independently swappable, injected collaborators at runtime — more flexible and testable."

3. What is a POJO, and how does it become a Spring Bean? Model answer: "A POJO is a Plain Old Java Object with no special framework requirements. It becomes a Bean only once the Spring container manages its lifecycle — typically via `@Component` and component scanning, or an explicit `@Bean` definition."

4. Explain method overloading vs overriding briefly. Model answer: "Overloading = same method name, different parameter list, resolved at compile time. Overriding = subclass redefines a parent's method with the same signature, resolved at runtime via polymorphism."

5. Why is immutability valued in Spring beans where possible? Model answer: "Immutable objects (via constructor injection + `final` fields) are inherently thread-safe and easier to reason about — critical since singleton beans are shared across all threads handling requests."

Round 2 — Spring Core

6. Explain IoC in your own words. Model answer: "Instead of my code creating and wiring objects with `new`, I declare what I need, and the Spring container creates, configures, and injects those dependencies for me."

7. Constructor vs field injection — defend your preferred choice. Model answer: Constructor injection — immutability, explicit required dependencies, fail-fast at startup if something's missing, and easy to unit test without Spring at all.

8. What happens if Spring finds two beans matching an `@Autowired` field with no qualifier? Model answer: "It throws `NoUniqueBeanDefinitionException` at startup — Spring won't guess. I'd resolve it with `@Primary` for a sensible default or `@Qualifier` for an explicit choice."

9. Explain the Bean lifecycle briefly. Model answer: Definition → Instantiation → Dependency Injection → `@PostConstruct` → ready for use → `@PreDestroy` on shutdown.

10. What's the risk of a singleton bean holding mutable instance state? Model answer: "Since one instance is shared across every thread handling requests, mutable state can cause race conditions — singleton beans should generally be stateless, or use thread-safe state handling."

Round 3 — Spring Boot

11. What does `@SpringBootApplication` actually do? Model answer: Combines `@SpringBootConfiguration`, `@EnableAutoConfiguration`, and `@ComponentScan` into one annotation.

12. How does auto-configuration avoid conflicting with a bean I define myself? Model answer: "Auto-configuration classes use `@ConditionalOnMissingBean` — they only apply their default if I haven't already defined my own bean of that type."

13. Why does Spring Boot embed the server instead of relying on external deployment? Model answer: "It makes the app self-contained and simpler to run/deploy consistently across environments — `java -jar app.jar` is enough, no separate Tomcat installation/configuration needed."

14. How would you manage different configs for dev vs production? Model answer: Spring Profiles — separate `application-{profile}.properties` files, activated via `spring.profiles.active`.

15. What's the danger of relying on `ddl-auto=update` in production? Model answer: "It's unpredictable for schema changes at scale and can lock tables or silently make unintended changes — production should use a controlled migration tool like Flyway instead."

Round 4 — REST API

16. Walk me through what happens when a client hits `POST /api/users`. Model answer: Full request-flow explanation — Tomcat receives it, DispatcherServlet routes it via HandlerMapping to the controller method, `@RequestBody` is deserialized by Jackson, controller delegates to service/repository, response flows back and is serialized to JSON.

17. Why shouldn't GET requests mutate server state? Model answer: "GET is expected to be safe and idempotent by REST convention — browsers, proxies, and crawlers can trigger GETs unexpectedly (prefetching, caching), so mutating state there risks unintended side effects."

18. When would you choose PATCH over PUT? Model answer: "When the client is only updating a subset of fields and I don't want to require/overwrite the rest of the resource."

19. Why use `ResponseEntity` instead of just returning the object? Model answer: "Returning a plain object always defaults to 200 OK — I need explicit control to return 201 on creation, 204 on deletion, or 404 when something's missing."

20. What's the difference between `@PathVariable` and `@RequestParam`, with an example each? Model answer: Uses `/users/{id}` (path variable — identifies the resource) vs `/users?name=Rachit` (query param — filters a collection).

Round 5 — Exception Handling

21. How do you avoid leaking stack traces to API clients? Model answer: Centralize handling in `@RestControllerAdvice`, catching specific exceptions and returning a clean, structured error response instead of letting exceptions propagate raw.

22. Local `@ExceptionHandler` vs global `@RestControllerAdvice` — when would you still use the local one? Model answer: "When an exception is genuinely specific to one controller's behavior and shouldn't be handled the same way elsewhere — otherwise I default to the global handler for consistency."

23. Why create a custom `UserNotFoundException` instead of throwing a generic `RuntimeException`? Model answer: "It documents intent clearly in code, and lets my global handler map it precisely to a 404, instead of a broad catch-all guessing at the right status."

24. What should a well-designed error response body contain? Model answer: Timestamp, status code, a clear message, and the request path — consistent shape across all failures.

25. How do you handle bean validation failures (`@Valid`) globally? Model answer: A dedicated `@ExceptionHandler(MethodArgumentNotValidException.class)` in the global advice, extracting field errors into a structured 400 response.

Round 6 — JDBC

26. Why is `PreparedStatement` preferred over `Statement`? Model answer: Prevents SQL injection (parameters are data, not SQL) and improves performance via query plan reuse.

27. What's the danger of not using try-with-resources with JDBC? Model answer: "Manually closing resources is error-prone — a missed close leaks database connections, eventually exhausting the pool under load."

28. Explain `executeQuery()` vs `executeUpdate()`. Model answer: `executeQuery()` for SELECT, returns a `ResultSet`; `executeUpdate()` for INSERT/UPDATE/DELETE, returns affected row count.

29. How would you detect and prevent SQL injection in a legacy codebase? Model answer: Audit for string-concatenated SQL, replace with `PreparedStatement` parameter binding everywhere user input touches a query.

30. What does the JDBC architecture look like? Model answer: Java app → JDBC API (interfaces) → vendor-specific driver → actual database.

Round 7 — Transactions

31. Explain ACID with a real example. Model answer: Bank transfer — atomicity ensures debit and credit both happen or neither does; consistency keeps account balances valid; isolation prevents concurrent transfers from seeing each other's half-finished state; durability ensures a committed transfer survives a crash.

32. What does `setAutoCommit(false)` change about JDBC's default behavior? Model answer: "By default every statement auto-commits immediately; disabling it lets me group multiple statements into one atomic transaction, committing or rolling back together."

33. When would you roll back a transaction manually? Model answer: "Whenever any step in a multi-step operation fails partway through, to avoid leaving the database in a half-updated, inconsistent state."

34. How does `@Transactional` in Spring compare to manual JDBC transaction management? Model answer: "It achieves the same goal declaratively — Spring wraps the method in an AOP proxy that begins/commits/rolls back the transaction automatically, instead of me writing `commit()`/`rollback()` by hand."

35. What's a scenario where isolation matters even within a single transaction? Model answer: Two concurrent transactions both reading and updating the same inventory row — proper isolation (or explicit locking) prevents lost updates.

Round 8 — Hibernate/JPA

36. Clarify: is Hibernate the same as JPA? Model answer: "No — JPA is a specification, a set of interfaces and annotations. Hibernate is one implementation of that specification, and the most widely used one."

37. What is dirty checking? Model answer: "Within an active persistence context, Hibernate tracks field changes on managed entities and automatically issues the corresponding UPDATE at commit — no explicit save call required."

38. Explain the entity lifecycle states. Model answer: Transient (new, unmanaged) → Managed (tracked in persistence context) → Detached (context closed) → Removed (scheduled for deletion).

39. What causes `LazyInitializationException`, and how do you fix it? Model answer: "Accessing a lazily-loaded association after its session/persistence context has closed. Fixed with `JOIN FETCH`, `@EntityGraph`, or returning DTOs assembled while the session is still open."

40. Why might Hibernate produce more SQL queries than expected for a simple list fetch? Model answer: "The N+1 problem — fetching a list of parents, then lazily triggering a separate query per parent for a related collection. Fixed with a fetch join."

Round 9 — Relationships

41. Explain `mappedBy` in a bidirectional `@OneToMany`/`@ManyToOne`. Model answer: "`mappedBy` goes on the inverse side (usually the `@OneToMany` side), pointing to the field on the owning entity (`@ManyToOne` side) that actually holds the foreign key — this avoids Hibernate creating a redundant join table."

42. Why is `@ManyToOne` typically the owning side of a relationship? Model answer: "Because the foreign key naturally lives on the 'many' side's table — each child row has one reference back to its parent."

43. Why does `@ManyToMany` require a join table? Model answer: "Neither side can store a single foreign key representing a many-to-many association, so a separate table with two FK columns is required to represent all the pairings."

44. What's the danger of `CascadeType.ALL` on a `@OneToMany`? Model answer: "It includes REMOVE — deleting the parent will delete all children too, which is dangerous if those children have independent significance and should instead be reassigned."

45. Difference between `CascadeType.REMOVE` and `orphanRemoval`? Model answer: "`CascadeType.REMOVE` deletes children when the parent is deleted; `orphanRemoval` deletes a child specifically when it's removed from the parent's in-memory collection, even if the parent itself isn't deleted."

Round 10 — Real-World Scenarios

46. Design the layered flow for a new "place order" feature end-to-end. Model answer: Walks through Controller (validates + delegates) → Service (`@Transactional`, orchestrates order creation + inventory update) → Repository (JPA persistence) → DB, emphasizing where business rules and transaction boundaries live.

47. Your API needs to support 10x more traffic next quarter. What would you look at first? Model answer: Connection pool sizing, query performance (N+1 issues, missing indexes), caching opportunities, and whether stateless design allows horizontal scaling of the service layer.

48. A junior developer put all business logic inside the Controller. How do you explain why that's a problem? Model answer: "It breaks separation of concerns — the controller becomes hard to test independently of HTTP, business logic can't be reused outside a web context, and the class grows unmanageable over time. It belongs in the Service layer."

49. How would you version a REST API without breaking existing clients? Model answer: URL versioning (`/api/v2/...`) or header-based versioning — never silently changing an existing endpoint's contract.

50. Final: Summarize, in under a minute, the full journey of a Spring Boot request from HTTP to database and back. Model answer: "Tomcat receives the raw HTTP request; DispatcherServlet routes it via HandlerMapping to the right controller method; the controller delegates to a service, which applies business logic inside a transaction; the service calls a Spring Data JPA repository; Spring Data JPA generates the call into Hibernate, which manages the persistence context and issues SQL through JDBC to the database; the result flows back up, and Jackson serializes the final response object into JSON before it's returned to the client."